

Artificial Neural Networks (ANN)

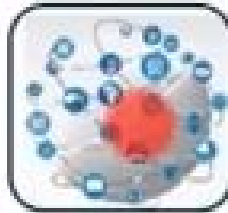
What is Neural Network

- Neural networks reflect the behavior of the human brain
- It helps computer programs to recognize patterns and solve common problems
- It is the fields of AI, machine learning, and deep learning.
- Neural networks, also known as artificial neural networks (ANNs) or simulated neural networks (SNNs)
- Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another.

Limitations Of Machine Learning

Are not useful while working with high dimensional data, that is where we have large number of inputs and outputs

Cannot solve crucial AI problems like NLP, Image recognition etc.

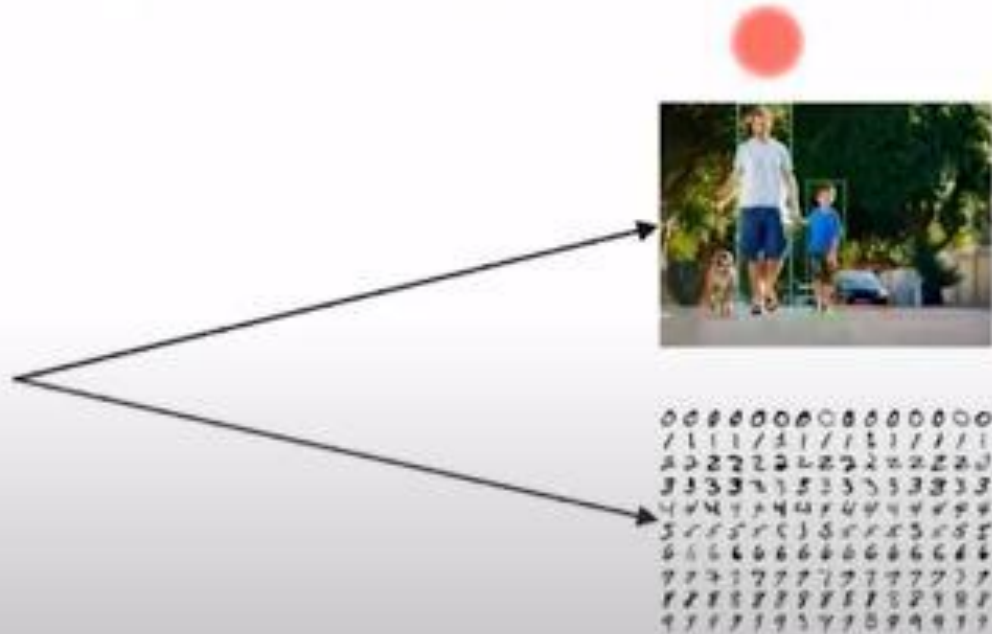


Limitations Of Machine Learning

Machine Learning

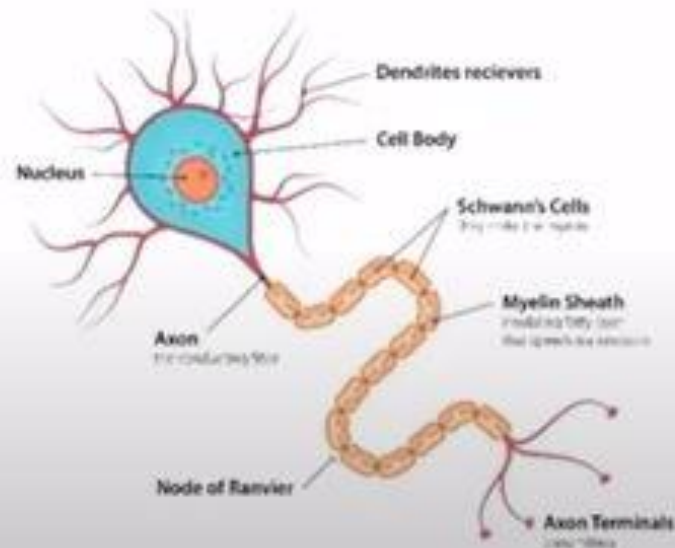
Limitations Of Machine Learning

- ❑ One of the big challenges with traditional Machine Learning models is a process called feature extraction.
- ❑ For complex problems such as object recognition or handwriting recognition, this is a huge challenge.

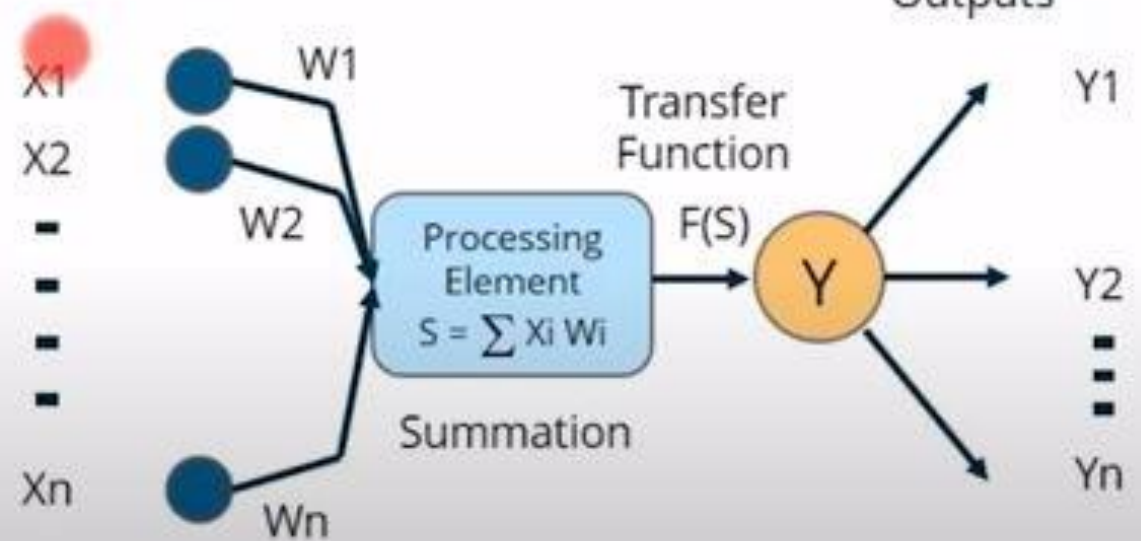


Deep Learning To The Rescue

- ❑ Deep Learning is implemented through Neural Networks.
- ❑ Motivation behind Neural Networks is the biological Neuron.

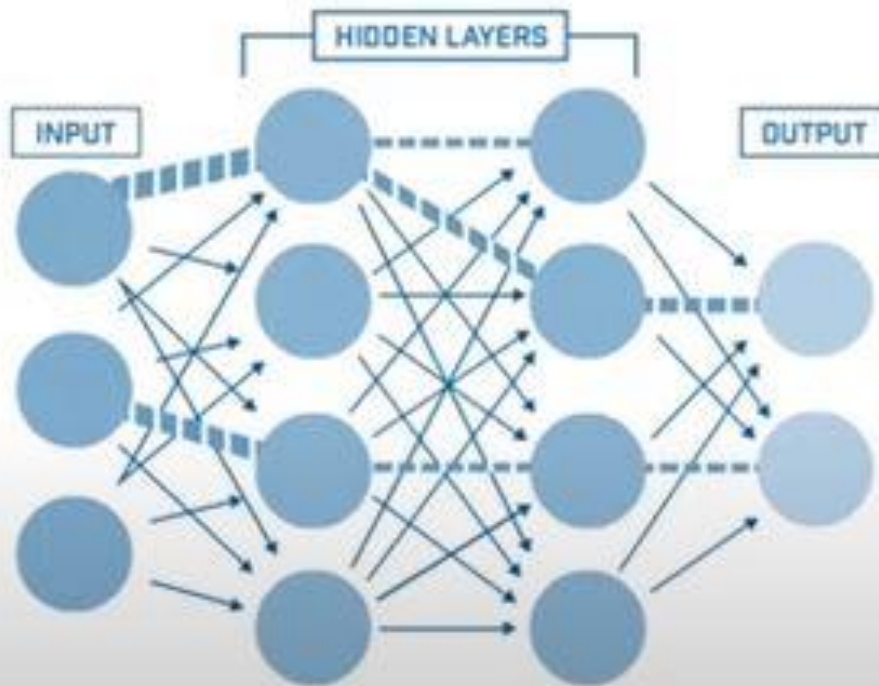


Neuron



Artificial Neural Network

Deep learning makes use of Artificial Neural Networks that behave similar to the Neural Networks in our brain.

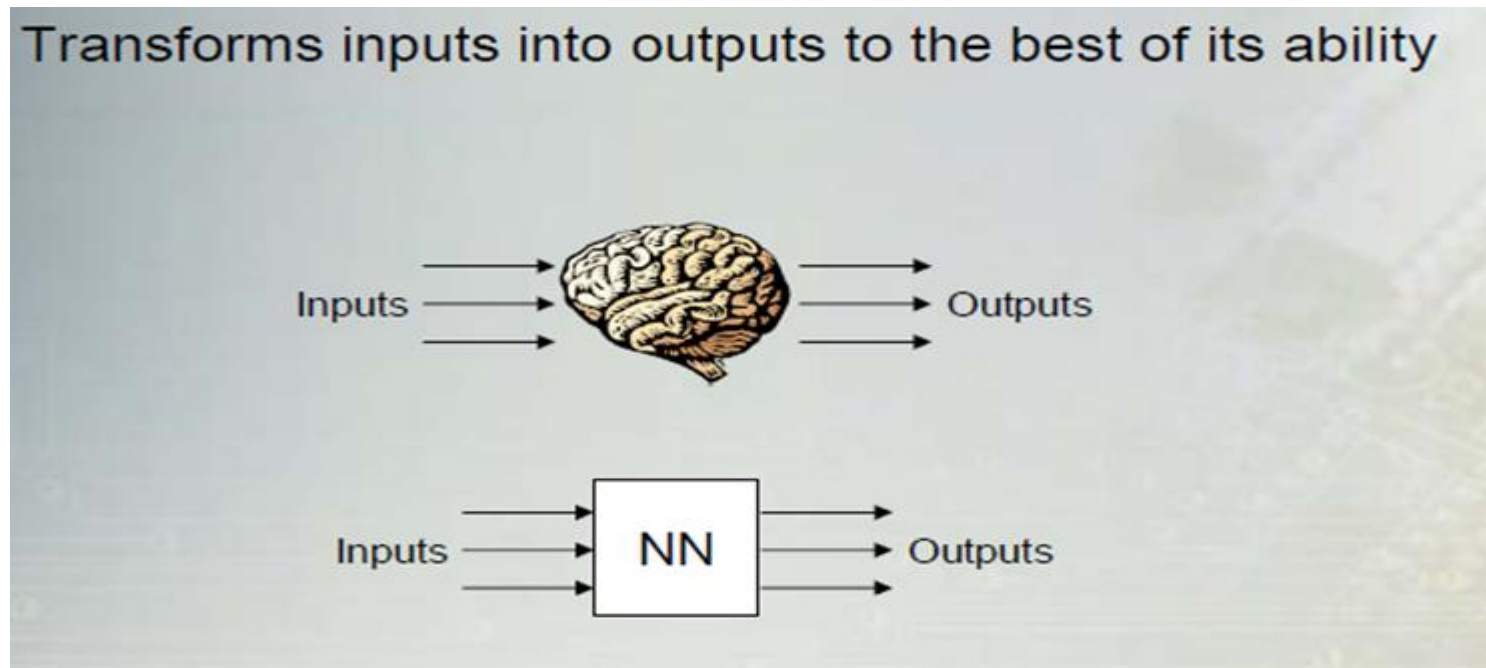


>

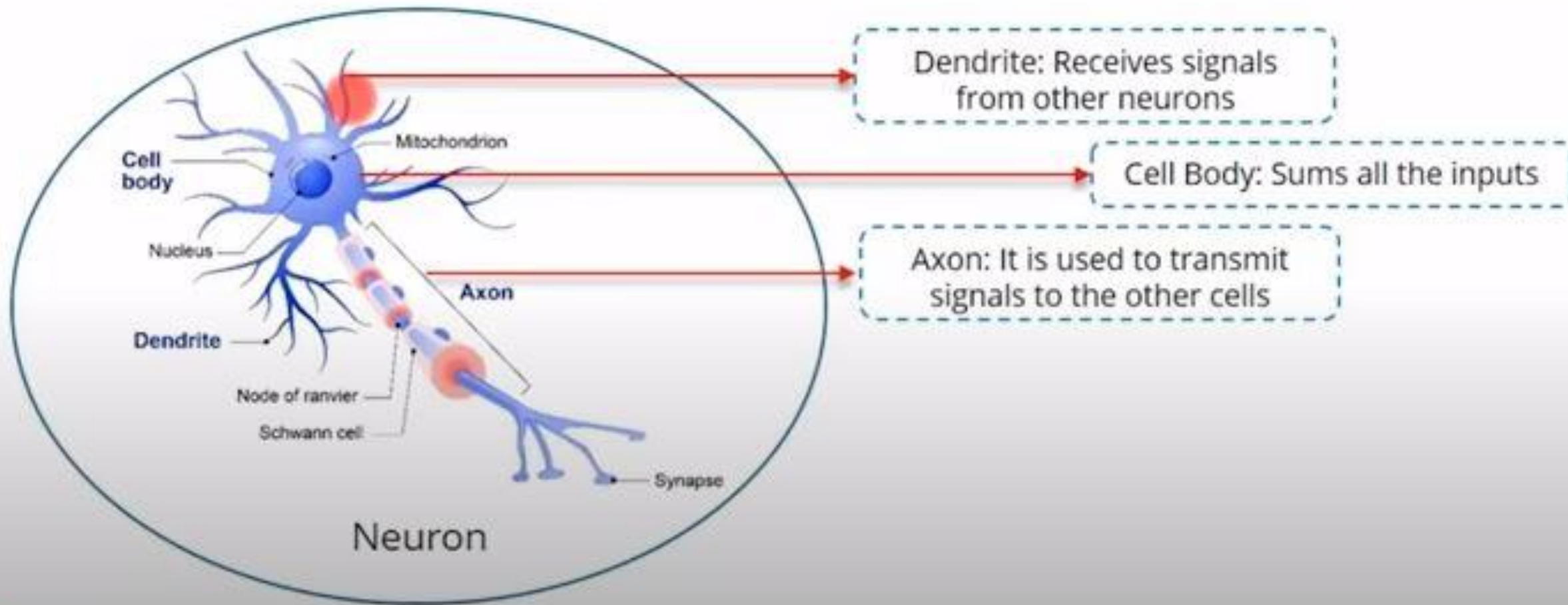


Artificial Neural Network

- Neural Networks are simplified models of the biological nervous system and have drawn motivation from the kind of computing performed by a brain.
- ANN in general is a highly interconnected network of a large no. of processing called 'Neurons' in an architecture inspired by the brain.
- The brain is a highly complex, nonlinear and parallel computer (Information Processing System).



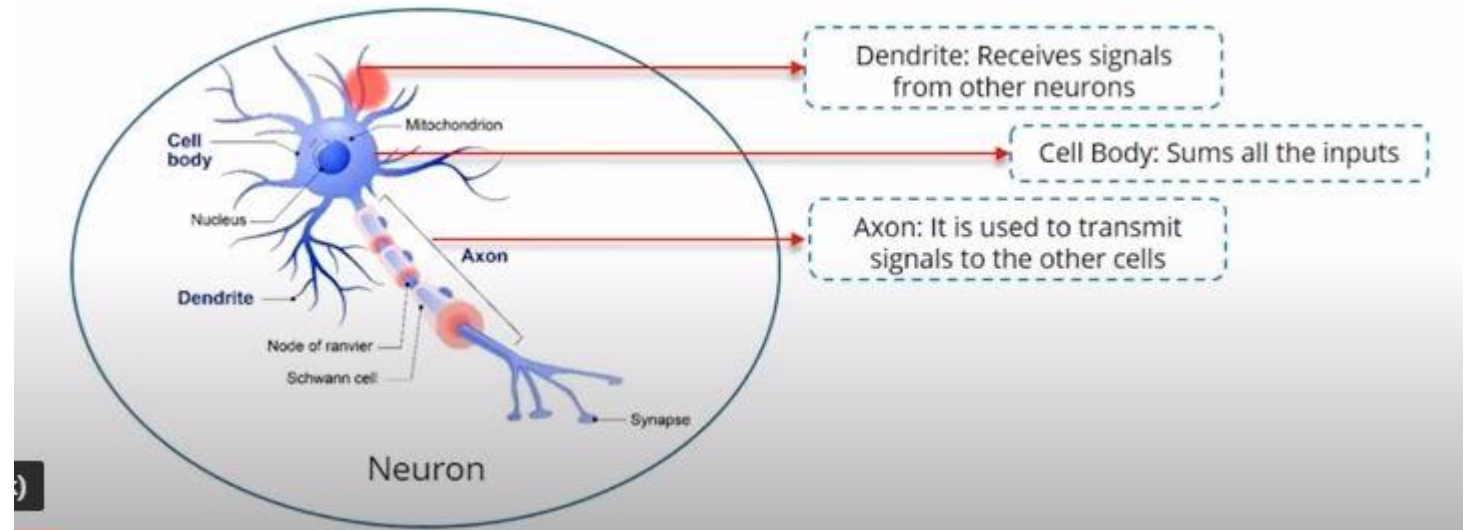
Background: ANN Vs Brain



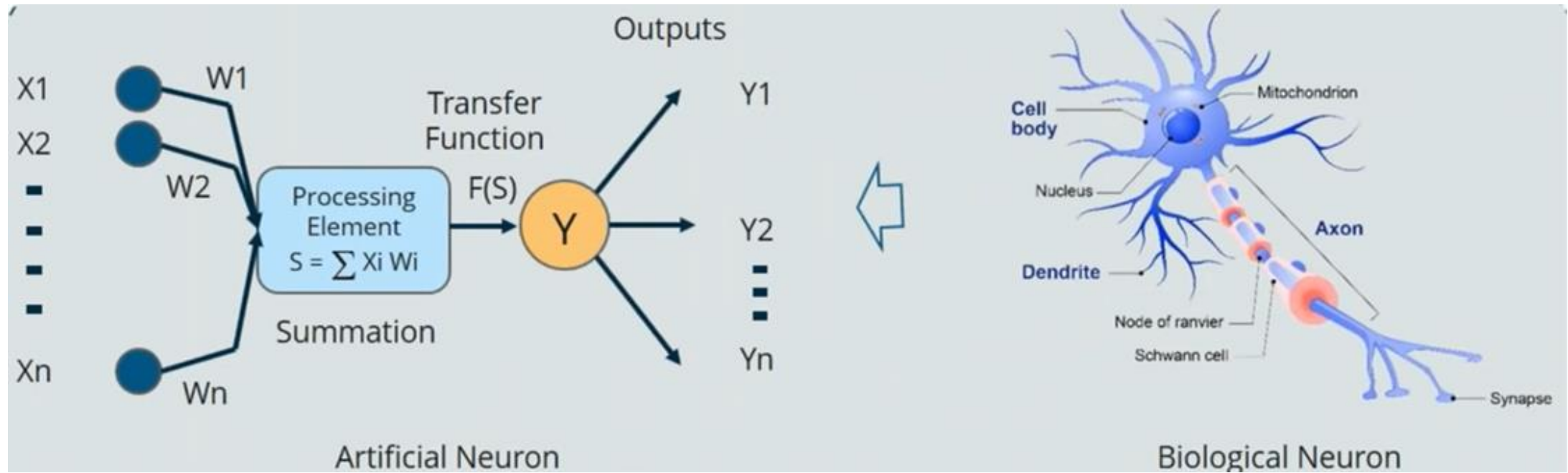
Background: ANN Vs Brain

In brain a neuron has three principal components:

1. **Dendrites:-** that carry electrical signals into the cell body.
2. **Cell Body:-** effectively sums and thresholds these incoming signals.
3. **Axon:-** is a single long fiber that carries the signal from the cell body out to other neurons.
4. The point of contact between an axon of one cell and a dendrite of another cell is called a '**synapse**'



Artificial Neuron v/s Biological Neuron



Background: ANN Vs Brain

ANN	Brain
It is simple (few neuron in connection)	It is complex (10^{11} Neurons and 10^{15} connections)
It is dedicated for specific purpose	It is generalized for all purpose
Response time is fast (it may be in Nanosecond)	Response time is slow (it may be in millisecond)
Design is regular	Design is arbitrary
Activities are synchronous	Activities are asynchronous

Deep Learning Applications



Self Driving Cars



Voice Controlled Assistance



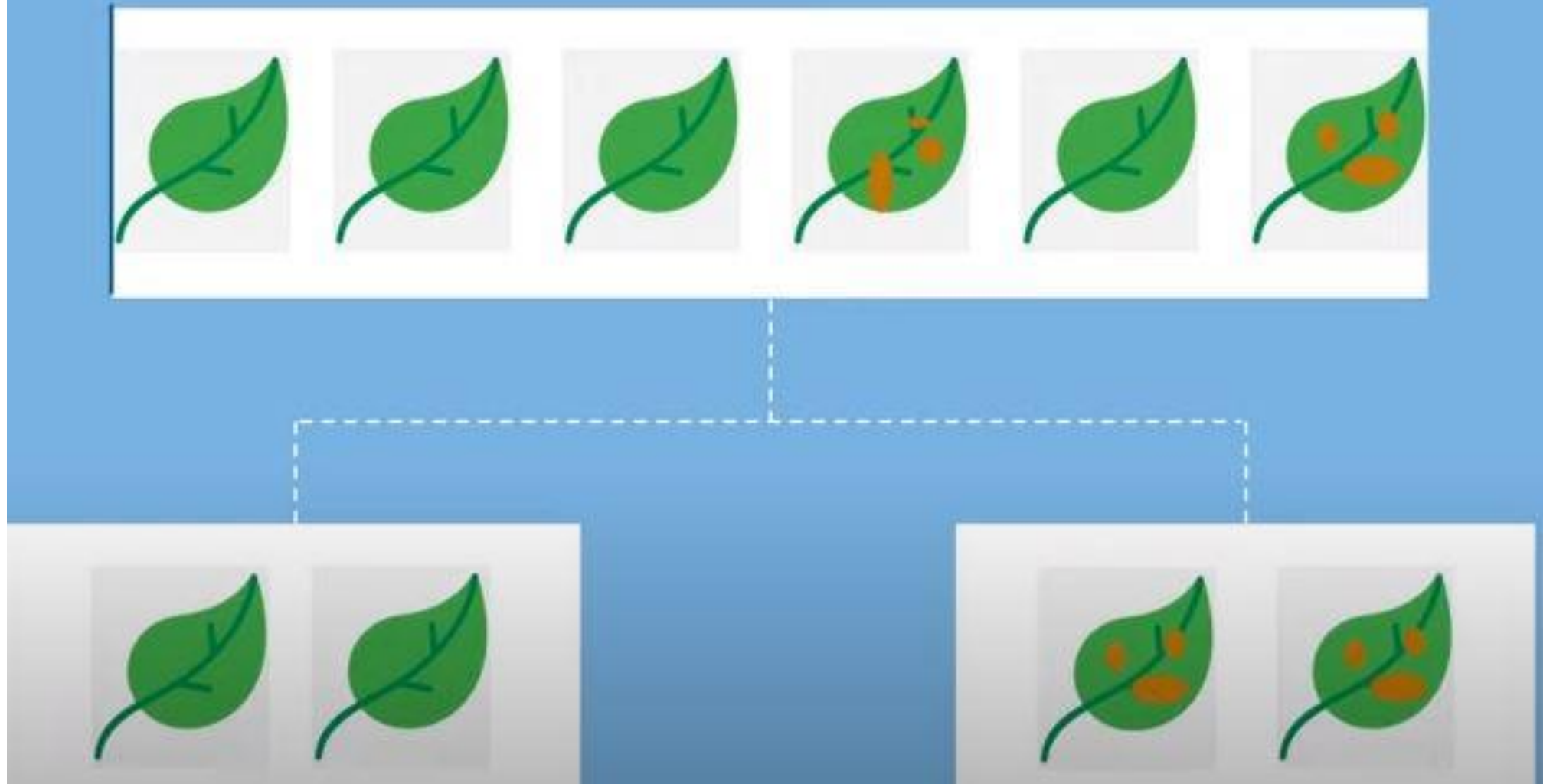
Automatic Image Caption Generation



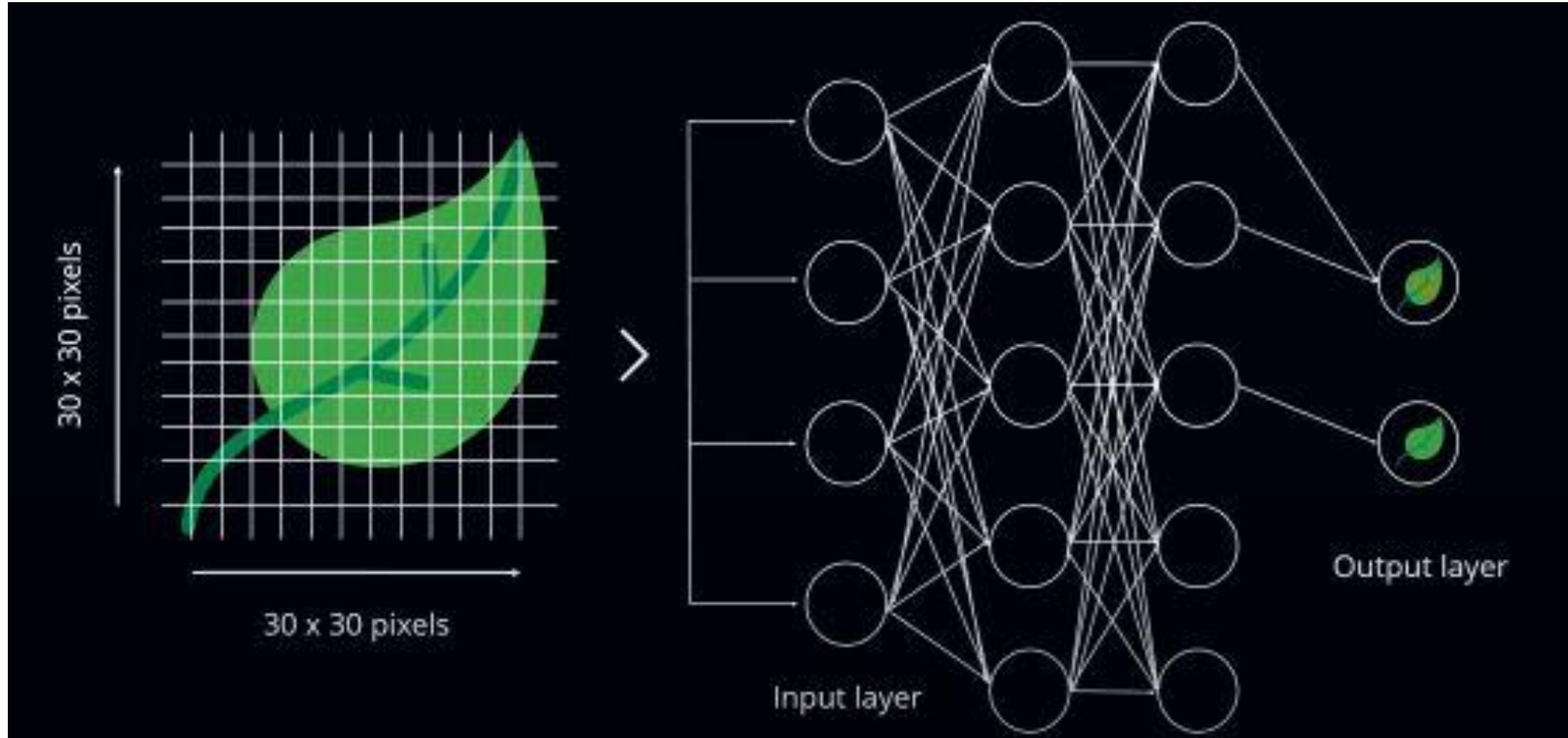
Automatic Machine Translation

Example:

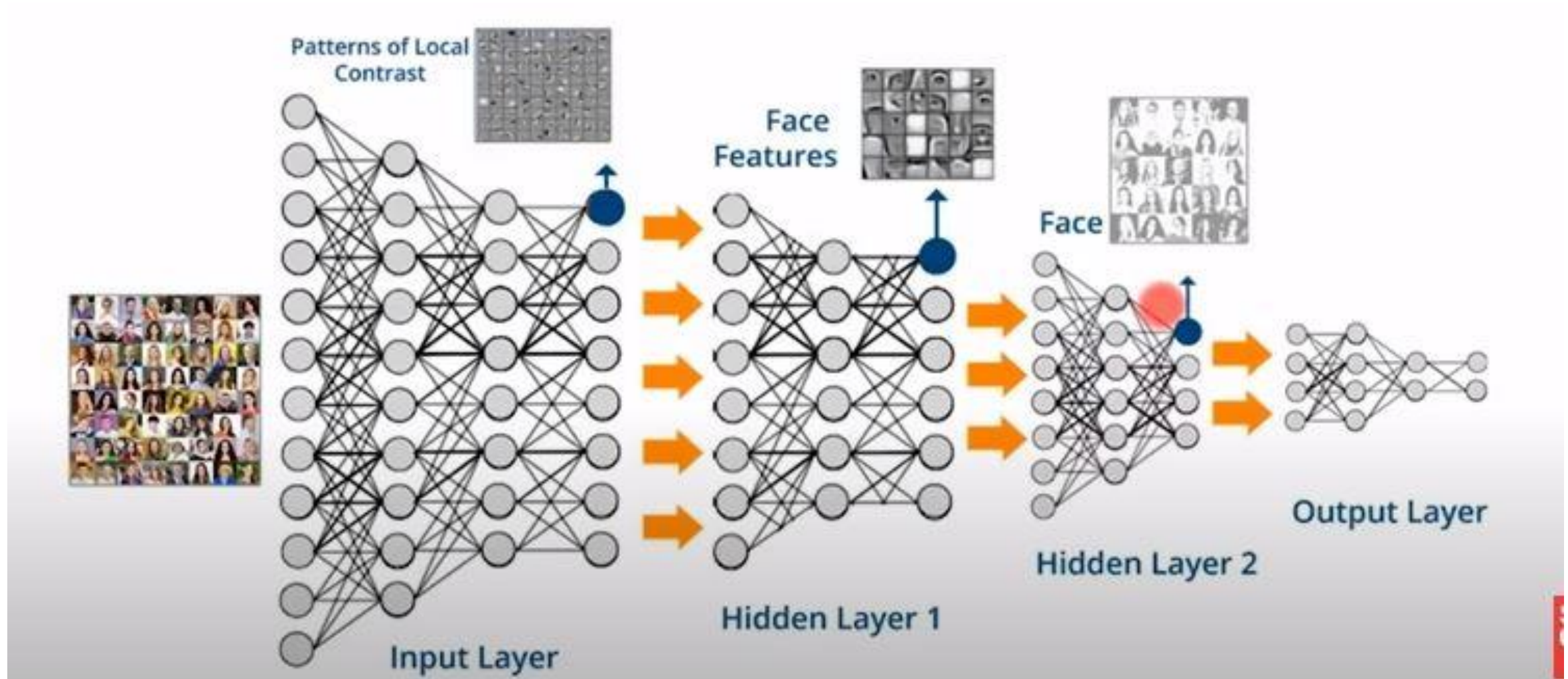
Problem Statement: Classify leaf images as either diseased or non-diseased.



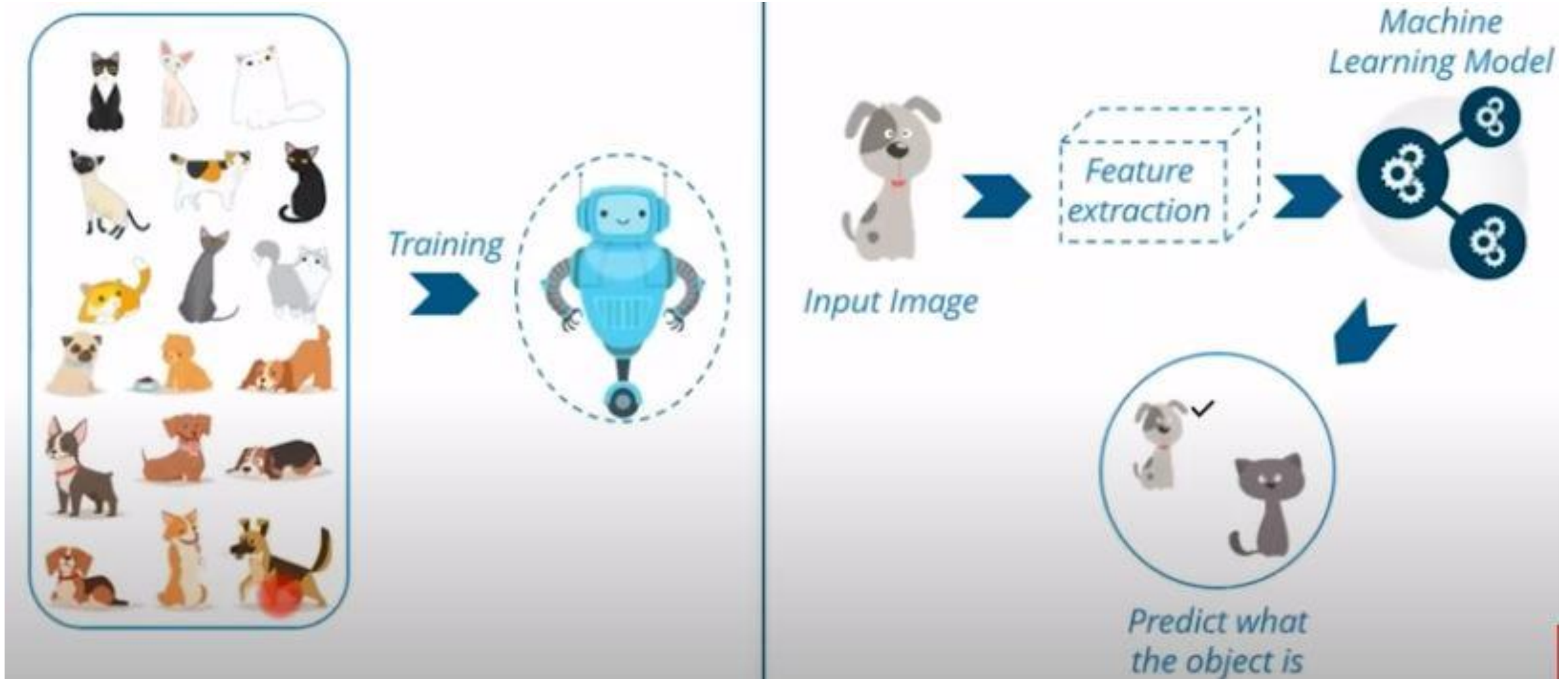
Example



Example:

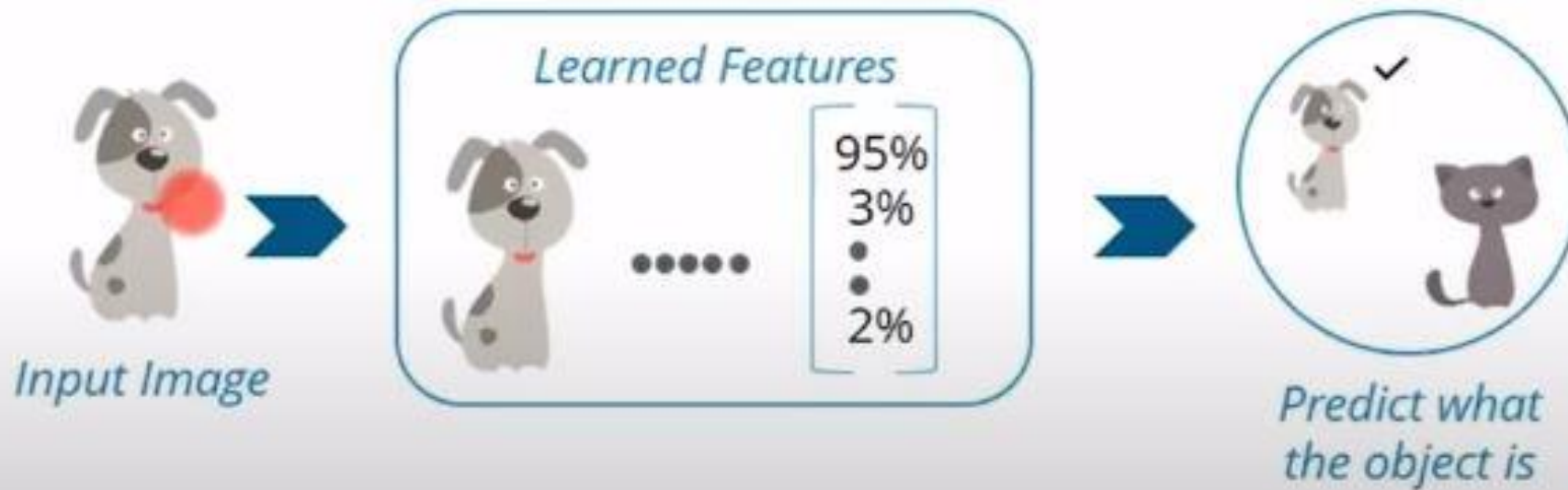


Example:



Example: Deep Learning/ ANN vs. ML

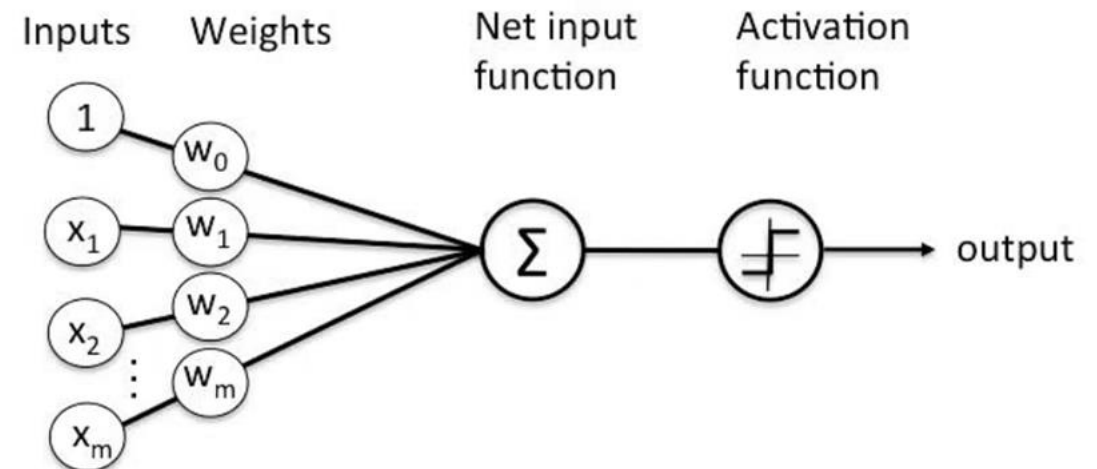
Deep Learning skips the manual steps of extracting features, you can directly feed images to the deep learning algorithm, which then predicts the object.



How Artificial Neural Network Works?

Perceptron

- Perceptron was introduced by Frank Rosenblatt in 1957.
- He proposed a Perceptron learning rule.
- There are two types of Perceptrons: Single layer and Multilayer.
- **Single layer** - Single layer perceptrons can learn only linearly separable patterns
- **Multilayer** - Multilayer perceptrons or feedforward neural networks with two or more layers have the greater processing power

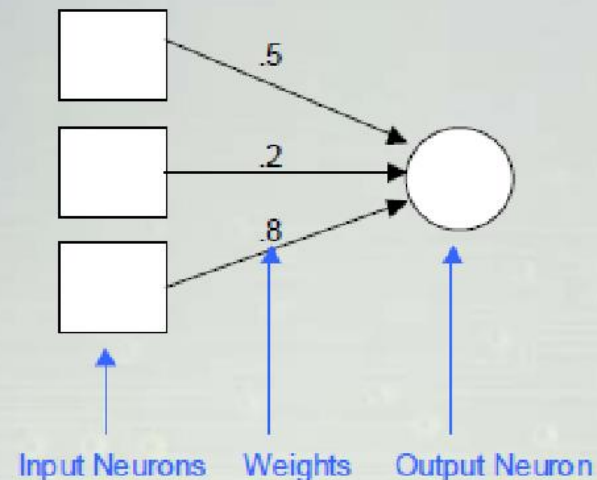


Perceptron

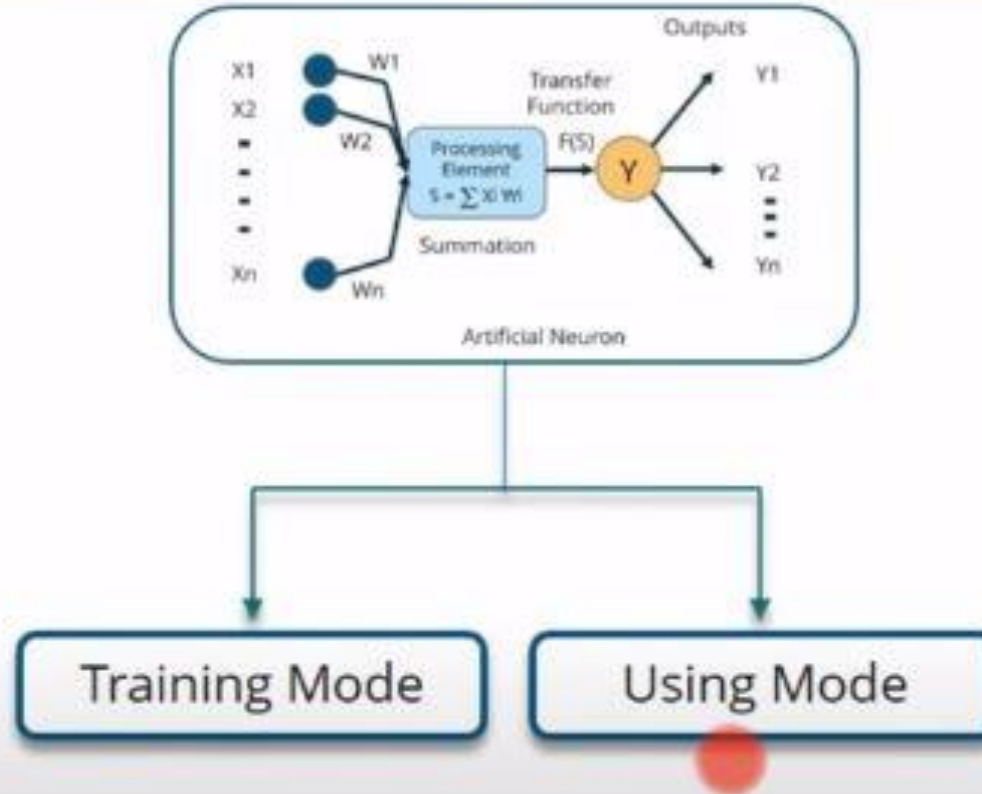
- A learning algorithm that self-arranges a network of artificial neurons to incorporate desired behaviors, the significance of the perceptron model enables machines to work efficiently in terms of binary classification
- While the single layer perceptrons can only classify inputs into classes with a single layer
- The multi-layer perceptron enables the model to conduct a classification of inputs with the help of more than one layer, making it suitable for more advanced and complex inputs.

Artificial Neuron/ Perceptron

- First neural network with the ability to learn
- Made up of only input neurons and output neurons
- Input neurons typically have two states: ON and OFF
- Output neurons use a simple threshold activation function
- In basic form, can only solve linear problems
 - ▶ Limited applications



Modes In Perceptron



In the training mode, the neuron can be trained to fire (or not), for particular input patterns



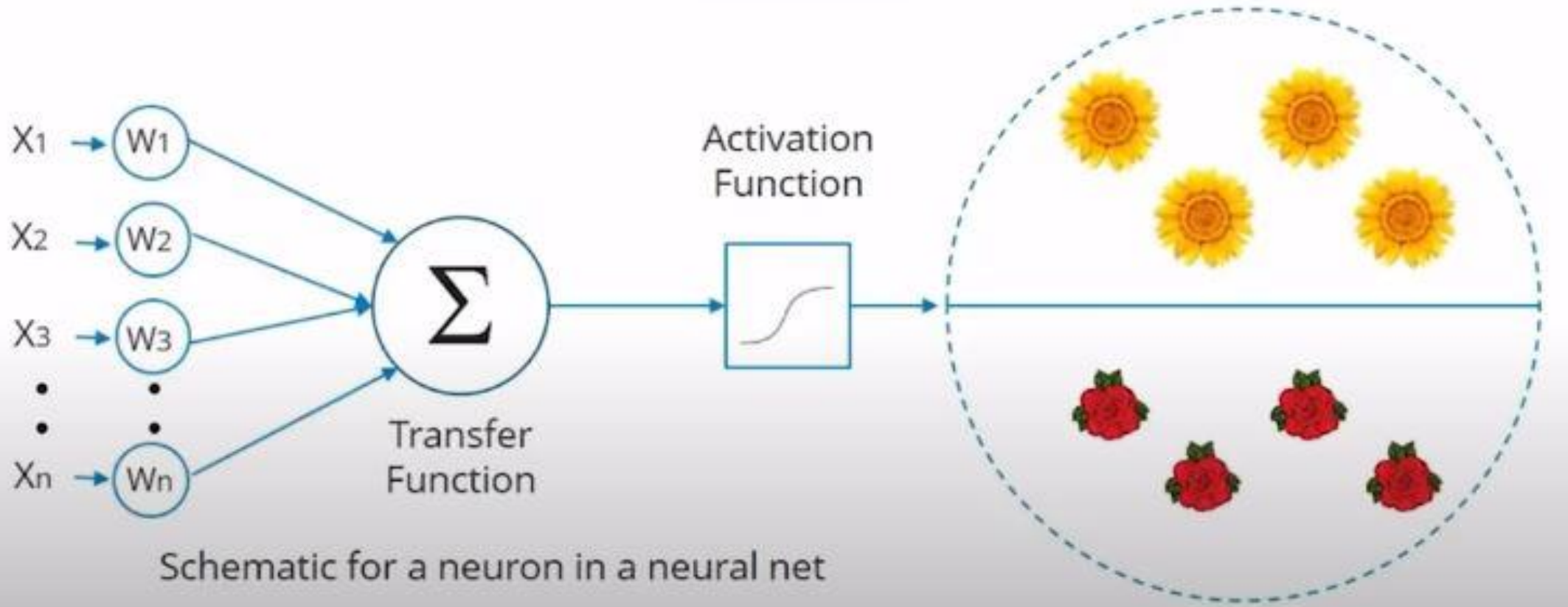
In the using mode, when a taught input pattern is detected at the input, its associated output becomes the current output

Modes in Perceptron

- Neural networks rely on training data to learn and improve their accuracy over time.
- However, once these learning algorithms are fine-tuned for accuracy, they are powerful tools in AI and ML
- Tasks in speech recognition or image recognition can take minutes versus hours when compared to the manual identification by human experts.
- One of the most well-known neural networks is Google's search algorithm.

Perceptron Learning Algorithm

To get started, I'll explain a type of artificial neuron called a *perceptron*.



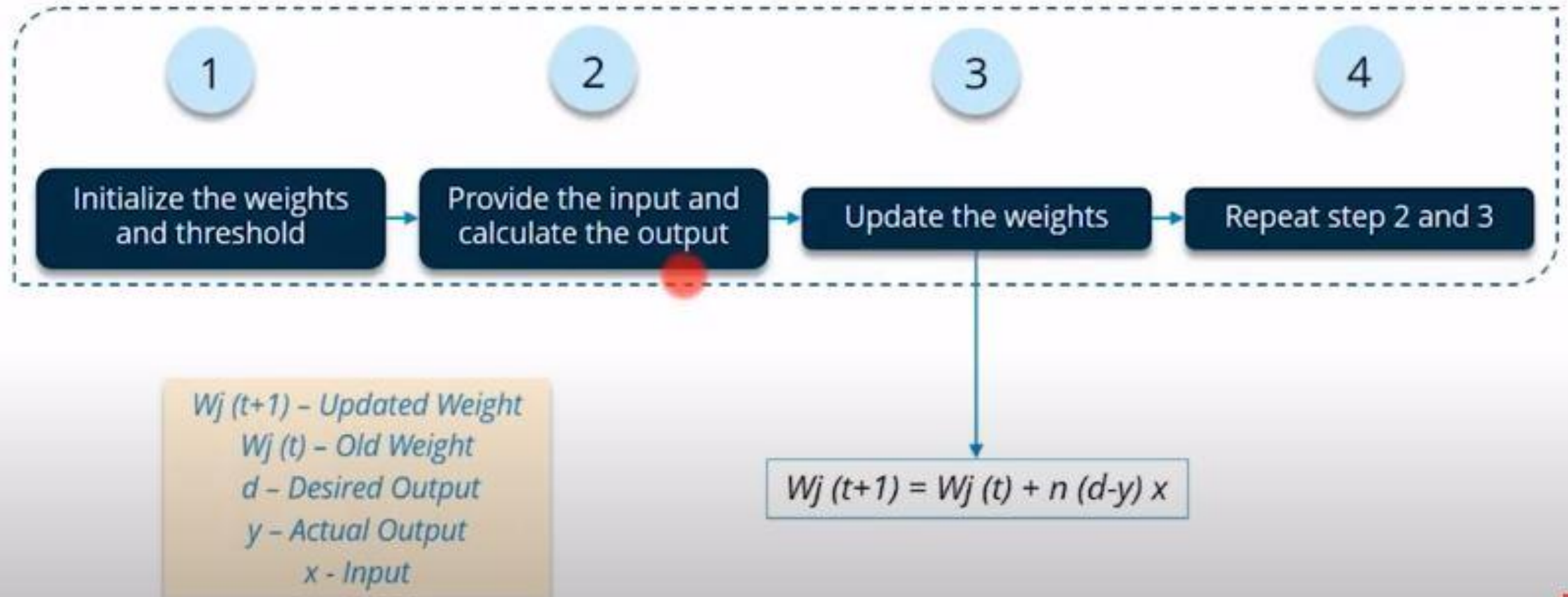
Perceptron Learning Algo.

- A neuron is a mathematical function modeled on the working of biological neurons
- It is an elementary unit in an artificial neural network
- One or more inputs are separately weighted
- Inputs are summed and passed through a nonlinear function to produce output
- Every neuron holds an internal state called activation signal
- Each connection link carries information about the input signal
- Every neuron is connected to another neuron via connection link

Perceptron Learning Algo.

- Artificial neural networks (ANNs) are comprised of a node layers, containing an input layer, one or more hidden layers, and an output layer.
- Each node, or artificial neuron, connects to another and has an associated weight and threshold.
- If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network.
- Otherwise, no data is passed along to the next layer of the network.
- The Perceptron Model in Machine Learning is a supervised learning algorithm that focuses on the linear binary classification of inputs.
- This algorithm, when it was primarily being worked upon, was intended to facilitate image recognition in machines.

Perceptron Learning Algorithm



Perceptron Function

- Perceptron is a function that maps its input “x,” which is multiplied with the learned weight coefficient; an output value “f(x)” is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\sum_{i=1}^m w_i x_i$$

- “w” = vector of real-valued weights
 - “b” = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
 - “x” = vector of input x values
 - “m” = number of inputs to the Perceptron
- The output can be represented as “1” or “0.”

Input of a Perceptron

- A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result.
- A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False.
- The summation function “ Σ ” multiplies all inputs of “x” by weights “w” and then adds them up as follows:

$$w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Output of a Perceptron

- Inputs: $x_1 \dots x_n$
- Output: $o(x_1 \dots x_n)$
- Weights: $w_i \Rightarrow$ contribution of input x_i to the Perceptron output;
- $w_0 \Rightarrow$ bias or threshold
- If $\sum w_i x_i > 0$, output is +1, else -1. The neuron gets triggered only when weighted input reaches a certain threshold value.
- An output of +1 specifies that the neuron is triggered. An output of -1 specifies that the neuron did not get triggered.
- “sgn” stands for sign function with output +1 or -1.

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

$$o(\vec{x}) = \text{sgn}(\vec{w} \cdot \vec{x})$$

$$\text{sgn}(y) = \begin{cases} 1 & \text{if } y > 0 \\ -1 & \text{otherwise} \end{cases}$$

Error in Perceptron

- In the Perceptron Learning Rule, the predicted output is compared with the known output.
- If it does not match, the error is propagated backward to allow weight adjustment to happen.

Perceptron Learning Algorithm

- Artificial Neural Networks can be viewed as weighted directed graphs in which artificial neurons are nodes, and directed edges with weights are connections between neuron outputs and neuron inputs.
- The Artificial Neural Network receives information from the external world in pattern and image in vector form.
- These inputs are designated by the notation $x(n)$ for n number of inputs.
- Each input is multiplied by its corresponding weights.
- Weights are the information used by the neural network to solve a problem.
- Typically weight represents the strength of the interconnection between neurons inside the Neural Network.

Perceptron Learning Algorithm

- The weighted inputs are all summed up inside the computing unit (artificial neuron).
- In case the weighted sum is zero, bias is added to make the output not- zero or to scale up the system response.
- Bias has the weight and input always equal to '1'.
- The sum corresponds to any numerical value ranging from 0 to infinity.
- To limit the response to arrive at the desired value, the threshold value is set up.
- For this, the sum is forward through an activation function.
- The activation function is set to the transfer function to get the desired output.

Where Do The Weights Come From?

- The weights in a neural network are the most important factor in determining its function
- Training is the act of presenting the network with some sample data and modifying the weights to better approximate the desired function
- There are two main types of training
 - ▶ Supervised Training
 - Supplies the neural network with inputs and the desired outputs
 - Response of the network to the inputs is measured
 - ◆ The weights are modified to reduce the difference between the actual and desired outputs

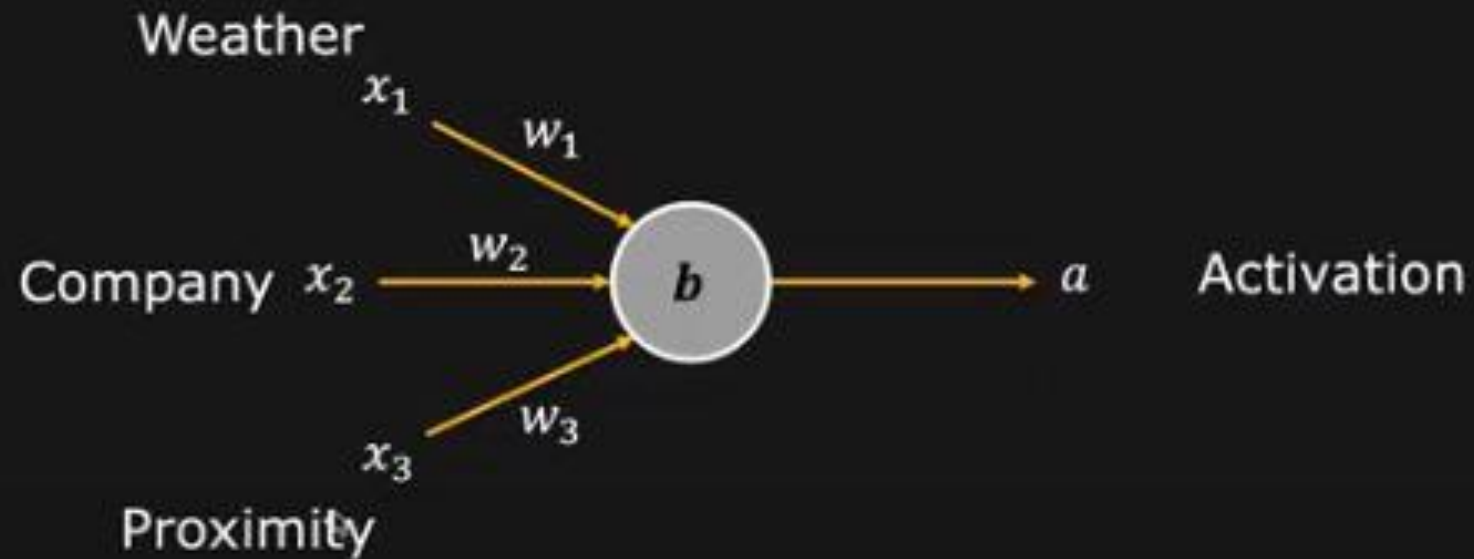
Where Do The Weights Come From?

► Unsupervised Training

- Only supplies inputs
- The neural network adjusts its own weights so that similar inputs cause similar outputs
 - ◆ The network identifies the patterns and differences in the inputs without any external assistance

Perceptron: Example

Will you go to the movies?



x_1 : Is the weather good?

x_2 : Do you have company?

x_3 : Is the theater nearby?

Assign Weights

W1 = Weight associated with input X1
W2 = Weight associated with input X2
W3 = Weight associated with input X3

W1 = 6, W2 = 2, W3 = 2



Threshold = 5

It will fire when weather is good and won't fire if weather is bad irrespective of the other inputs

W1 = 6, W2 = 2, W3 = 2

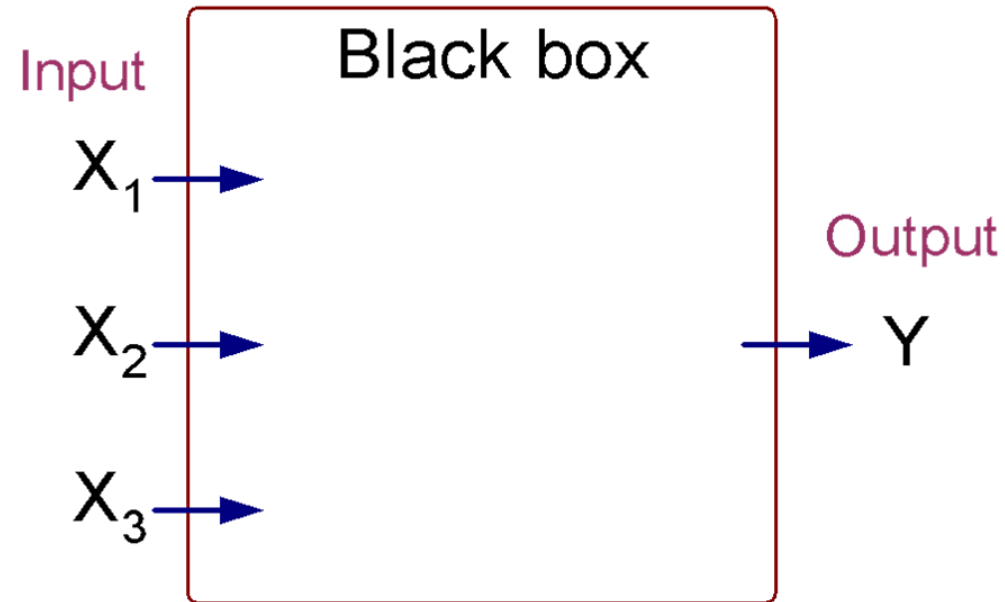


Threshold = 3

It will fire when either x1 is high or the other two inputs are high

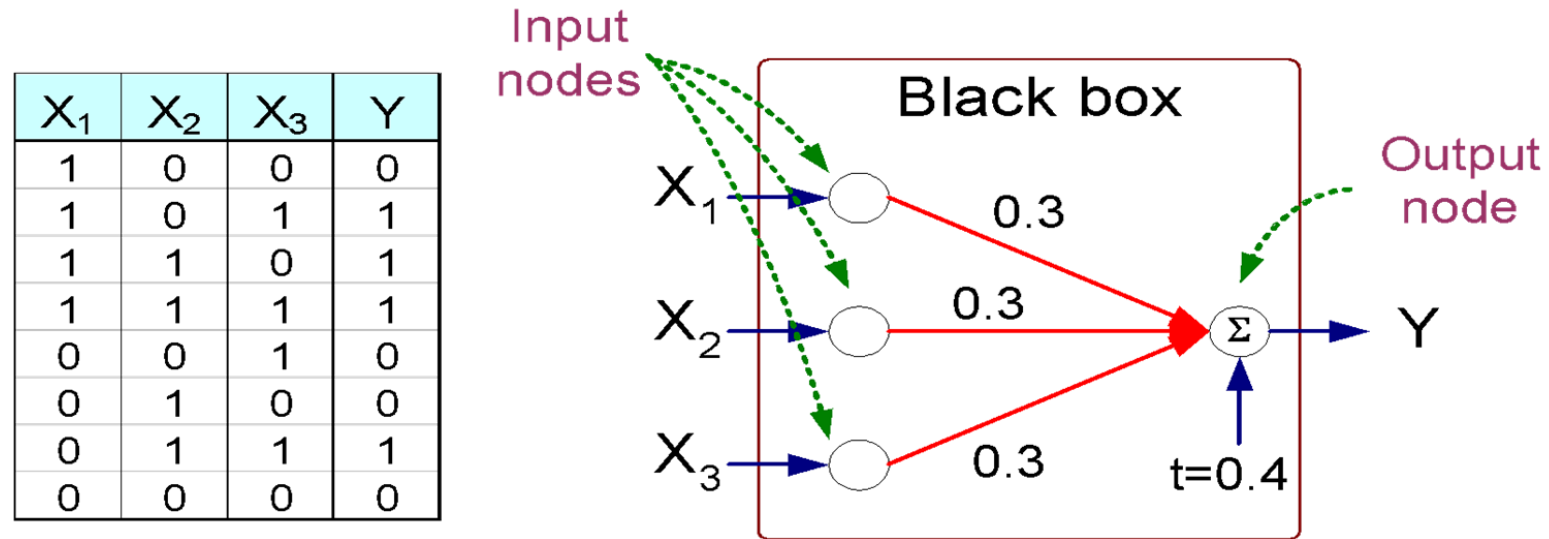
Example : Perceptron Learning Algorithm

X_1	X_2	X_3	Y
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	0
0	1	0	0
0	1	1	1
0	0	0	0



Output Y is 1 if at least two of the three inputs are equal to 1.

Example : Perceptron Learning Algorithm

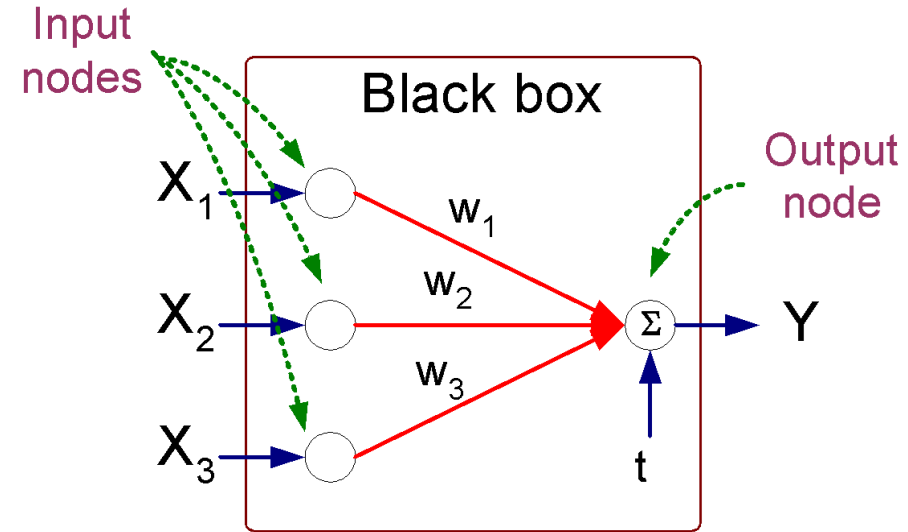


$$Y = I(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4 > 0)$$

$$\text{where } I(z) = \begin{cases} 1 & \text{if } z \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

Example : Perceptron Learning Algorithm

- Model is an assembly of inter-connected nodes and weighted links
- Output node sums up each of its input value according to the weights of its links
- Compare output node against some threshold t



Perceptron Model

$$Y = I\left(\sum_i w_i X_i - t\right)$$

Given the net input I_j to unit j , then O_j , the output of unit j , is computed as,

$$O_j = \frac{1}{1 + e^{-I_j}}.$$

This function is also referred to as a *squashing function*, because it maps a large input domain onto the smaller range of 0 to 1.

Training A Neural Network

The most common deep learning algorithm for supervised training of the multi-layer perceptrons is known as backpropagation. In it, after the weighted sum of inputs and passing through the activation function we propagate backwards and update the weights to reduce the error (desired output - model output). Consider the below example:

Input	Desired Output
0	0
1	2
2	4

Training A Neural Network

Now, we will see the error (Absolute and Square)

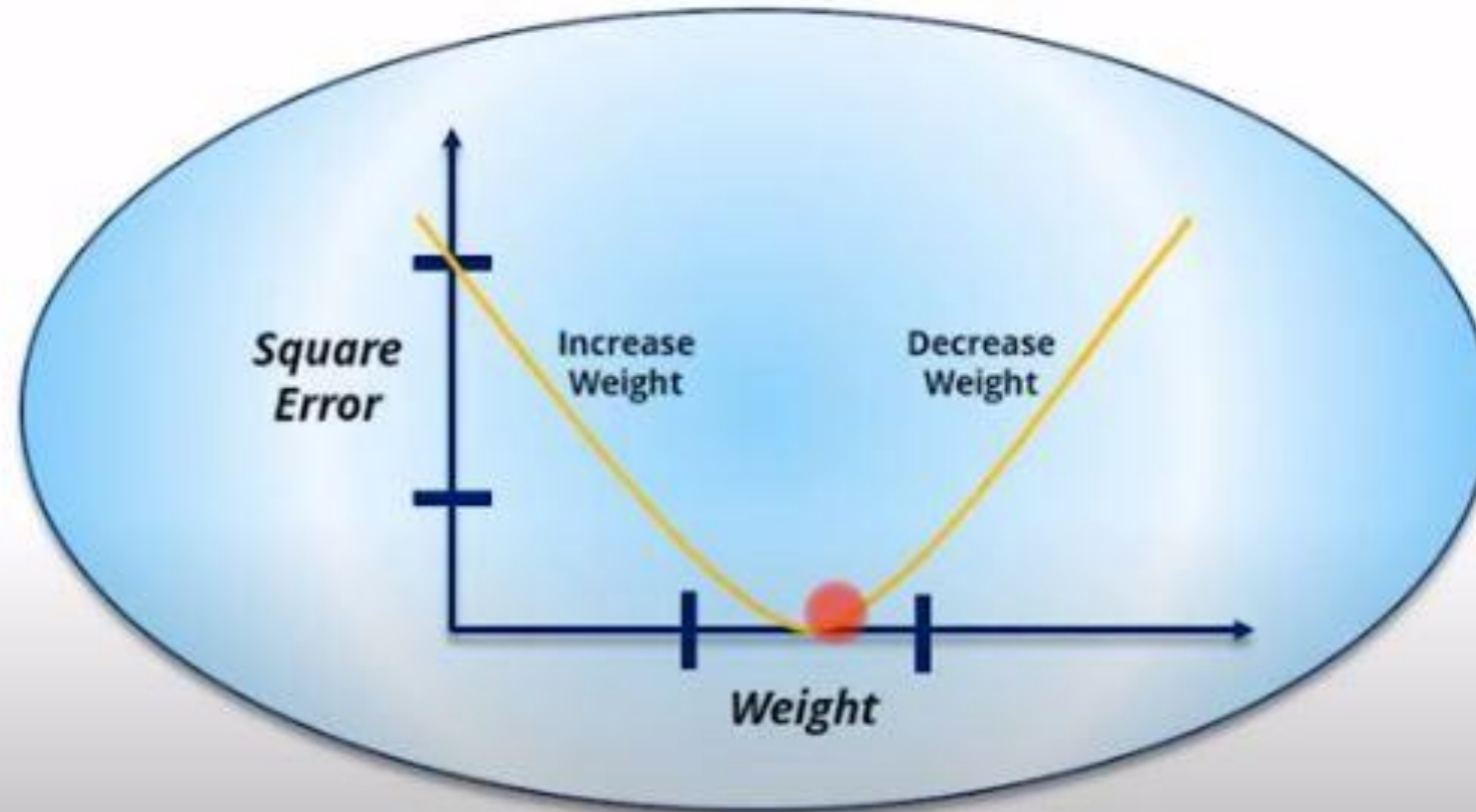
Input	Desired Output	Model Output (W=3)	Absolute Error	Square Error
0	0	0	0	0
1	2	3	1	1
2	4	6	2	4

Training A Neural Network

Still there is error, but we can see that error has increased

Input	Desired Output	Model Output (W=3)	Absolute Error	Square Error	Model Output (W=4)	Square Error
0	0	0	0	0	0	0
1	2	3	1	1	4	4
2	4	6	2	4	8	16

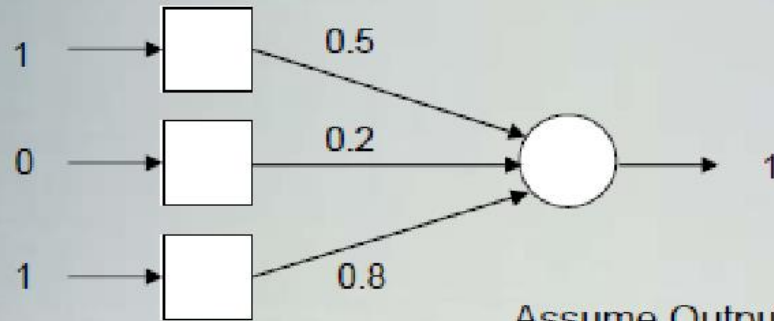
Training A Neural Network



Example : Perceptron Learning Algorithm

- Uses supervised training
- If the output is not correct, the weights are adjusted according to the formula:

$$\blacksquare W_{\text{new}} = W_{\text{old}} + \alpha(\text{desired} - \text{output}) * \text{input} \quad \alpha \text{ is the learning rate}$$



$$1 * 0.5 + 0 * 0.2 + 1 * 0.8 = 1.3$$

Assuming Output Threshold = 1.2

$$1.3 > 1.2$$

Assume Output was supposed to be 0
→ update the weights

Assume $\alpha = 1$

$$W_{1\text{new}} = 0.5 + 1 * (0 - 1) * 1 = -0.5$$

$$W_{2\text{new}} = 0.2 + 1 * (0 - 1) * 0 = 0.2$$

$$W_{3\text{new}} = 0.8 + 1 * (0 - 1) * 1 = -0.2$$

Learning Rate : Training Epochs

- The amount that the weights are updated during training is referred to as the step size or the “learning rate.”
- Specifically, the learning rate is a configurable hyperparameter used in the training of neural networks that has a small positive value, often in the range between 0.0 and 1.0.
- The learning rate controls how quickly the model is adapted to the problem. Smaller learning rates require more training epochs given the smaller changes made to the weights each update, whereas larger learning rates result in rapid changes and require fewer training epochs.
- A learning rate that is too large can cause the model to converge too quickly to a suboptimal solution, whereas a learning rate that is too small can cause the process to get stuck.
- The challenge of training deep learning neural networks involves carefully selecting the learning rate. It may be the most important hyperparameter for the model

Application of ANN

- Facial Recognition
- Handwriting Recognition
- Stock-Exchange Prediction
- Traveling issues of sales professionals
- Image compression
- Social Media
- Aerospace
- Defence
- Weather Forecasting
- Healthcare

Single-layer vs Multi-layer ANNs

- A single-layer neural network represents the most simple form of neural network, in which there is only one layer of input nodes that send weighted inputs to a subsequent layer of receiving nodes, or in some cases, one receiving node.
- A single-layer neural network was called a “perceptron.”
- A perceptron will either send a signal, or not, based on the weighted inputs.
- Another type of single-layer neural network is the single-layer binary linear classifier, which can isolate inputs into one of two categories.
- Single-layer neural networks can also be thought of as part of a class of feedforward neural networks, where information only travels in one direction, through the inputs, to the output.

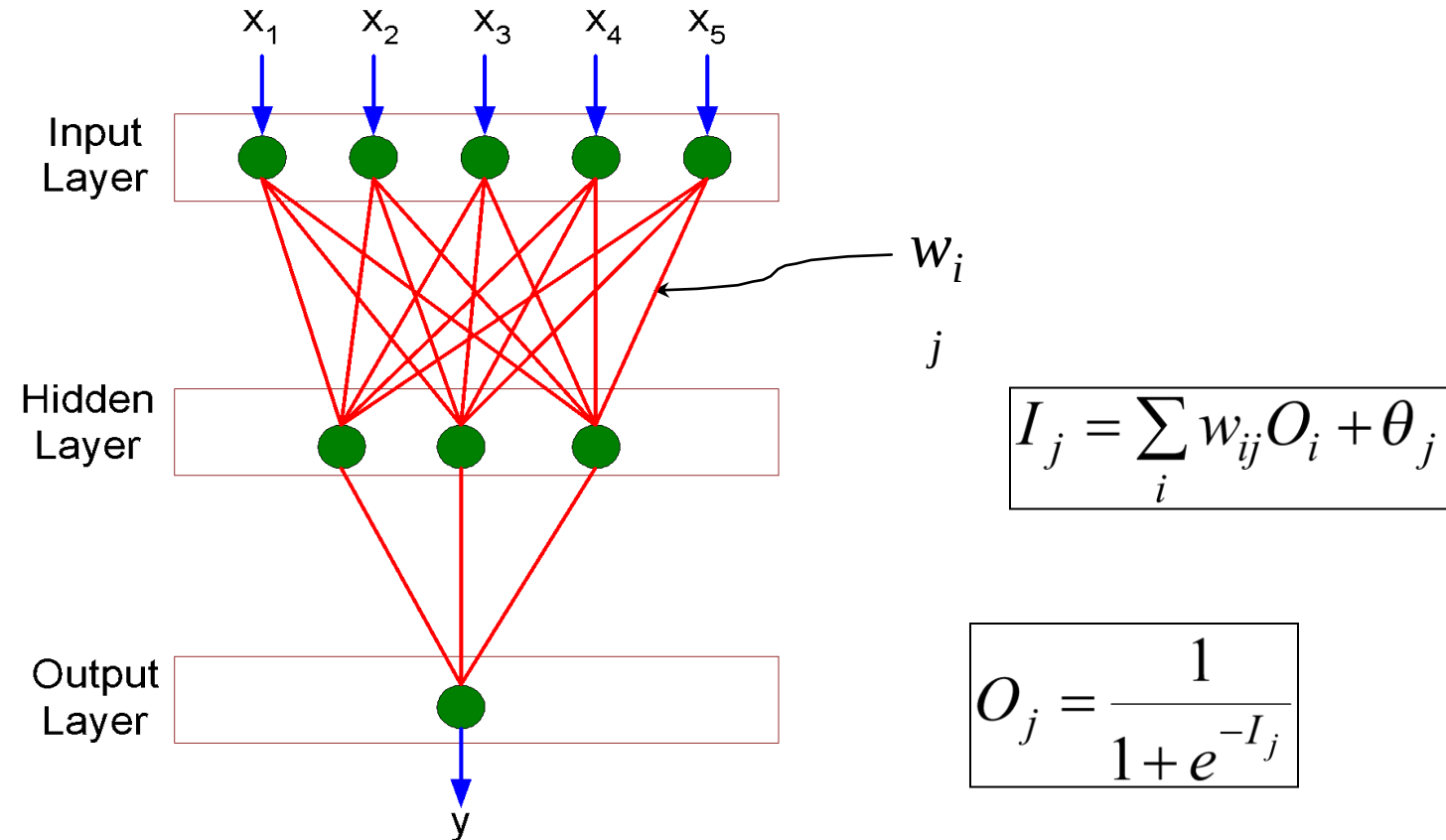
Multi-layer ANNs (MLP)

- Deep Learning deals with training multi-layer artificial neural networks, also called Deep Neural Networks.
- Today it is a research topic for many leading firms like Google, Facebook, and Microsoft which invest heavily in applications using deep neural networks.
- A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).
- If it has more than 1 hidden layer, it is called a deep ANN.
- The number of layers and the number of neurons are referred to as hyperparameters of a neural network, and these need tuning.
- The weight adjustment training is done via backpropagation.
- Deeper neural networks are better at processing data.

Feed Forward

- A feed-forward neural network is a biologically inspired classification algorithm.
- It consists of a number of simple neuron-like processing units, organized in layers and every unit in a layer is connected with all the units in the previous layer.
- They are known by many different names, such as 'multilayer perceptrons' (MLP).
- The goal of a feedforward network is to approximate some function f .
- For example, for a classifier, $y = f(x)$ maps an input x to a category y .
- A feedforward network defines a mapping $y = f(x; \theta)$ and learns the value of the parameters θ that result in the best function approximation
- Feed-forward neural networks have been widely used for a variety of function approximation tasks.

A Multi-Layer Feed-Forward Neural Network

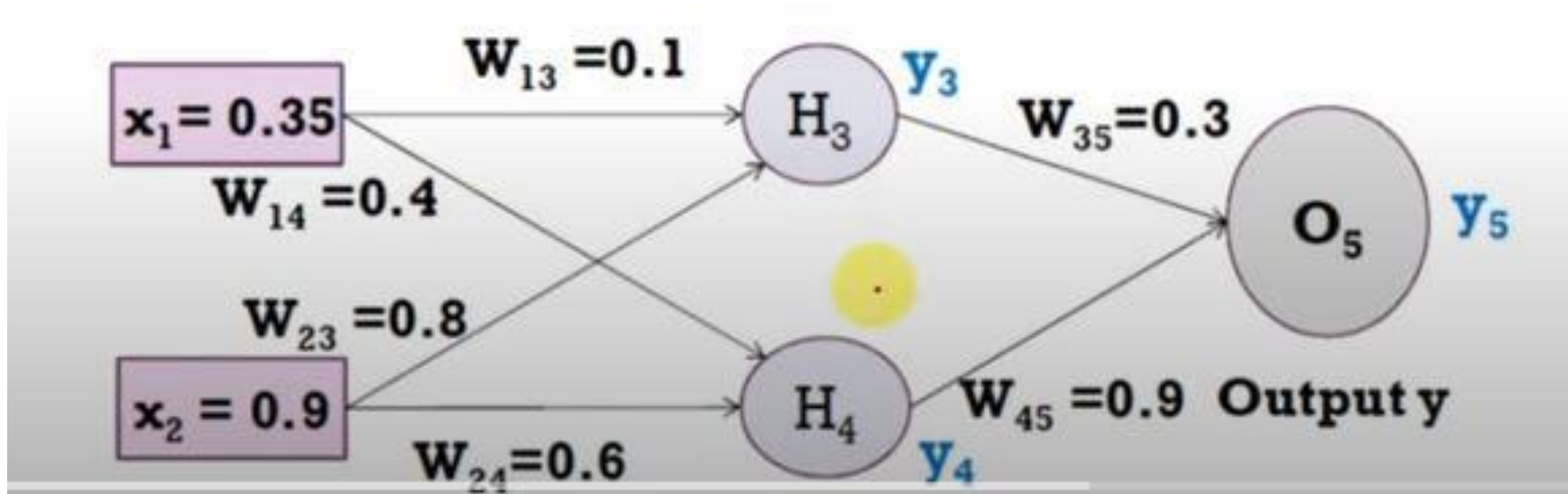


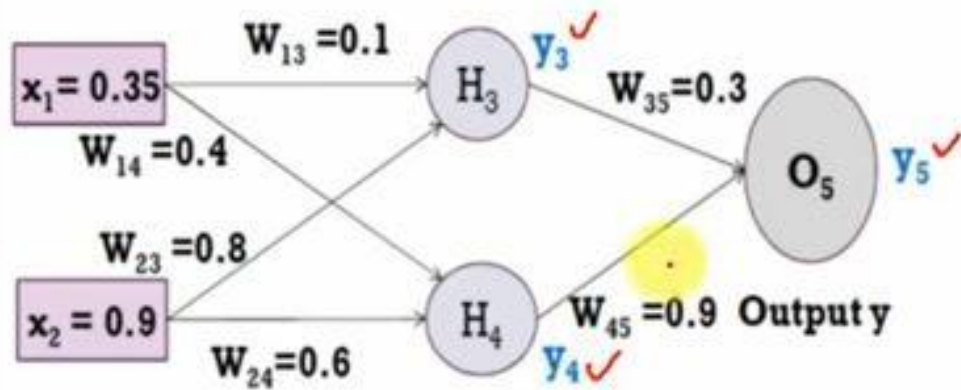
How A Multi-Layer Neural Network Works?

- The **inputs** to the network correspond to the attributes measured for each training tuple
- Inputs are fed simultaneously into the units making up the **input layer**
- They are then weighted and fed simultaneously to a **hidden layer**
- The number of hidden layers is arbitrary, although usually only one
- The weighted outputs of the last hidden layer are input to units making up the **output layer**, which gives out the network's prediction
- The network is **feed-forward** in that none of the weights cycles back to an input unit or to an output unit of a previous layer
- From a statistical point of view, networks perform **nonlinear regression**: Given enough hidden units and enough training samples, they can closely approximate any function

Defining a Network Topology

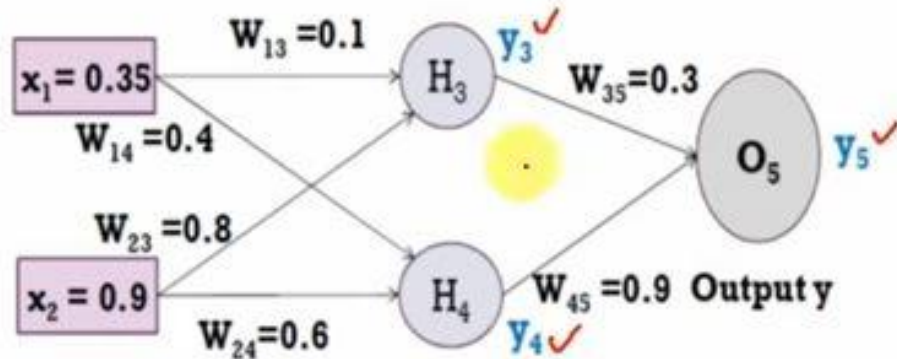
- First decide the **network topology**: # of units in the *input layer*, # of *hidden layers* (if > 1), # of units in *each hidden layer*, and # of units in the *output layer*
- Normalizing the input values for each attribute measured in the training tuples to [0.0—1.0]
- One **input** unit per domain value
- **Output**, if for classification and more than two classes, one output unit per class is used
- Once a network has been trained and its accuracy is **unacceptable**, repeat the training process with a *different network topology* or a *different set of initial weights*





- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$



- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$a_1 = (w_{13} * x_1) + (w_{23} * x_2) \\ = (0.1 * 0.35) + (0.8 * 0.9) = 0.755 \\ y_3 = f(a_1) = 1 / (1 + e^{-0.755}) = 0.68$$

$$a_2 = (w_{14} * x_1) + (w_{24} * x_2) \\ = (0.4 * 0.35) + (0.6 * 0.9) = 0.68 \\ y_4 = f(a_2) = 1 / (1 + e^{-0.68}) = 0.6637$$

$$Y5 = 0.69$$

$$\text{Error} = Y_{\text{target}} - Y5 = -0.19$$

How to solve Error??

Learning Algorithms:

Back propagation for classification

What is backpropagation

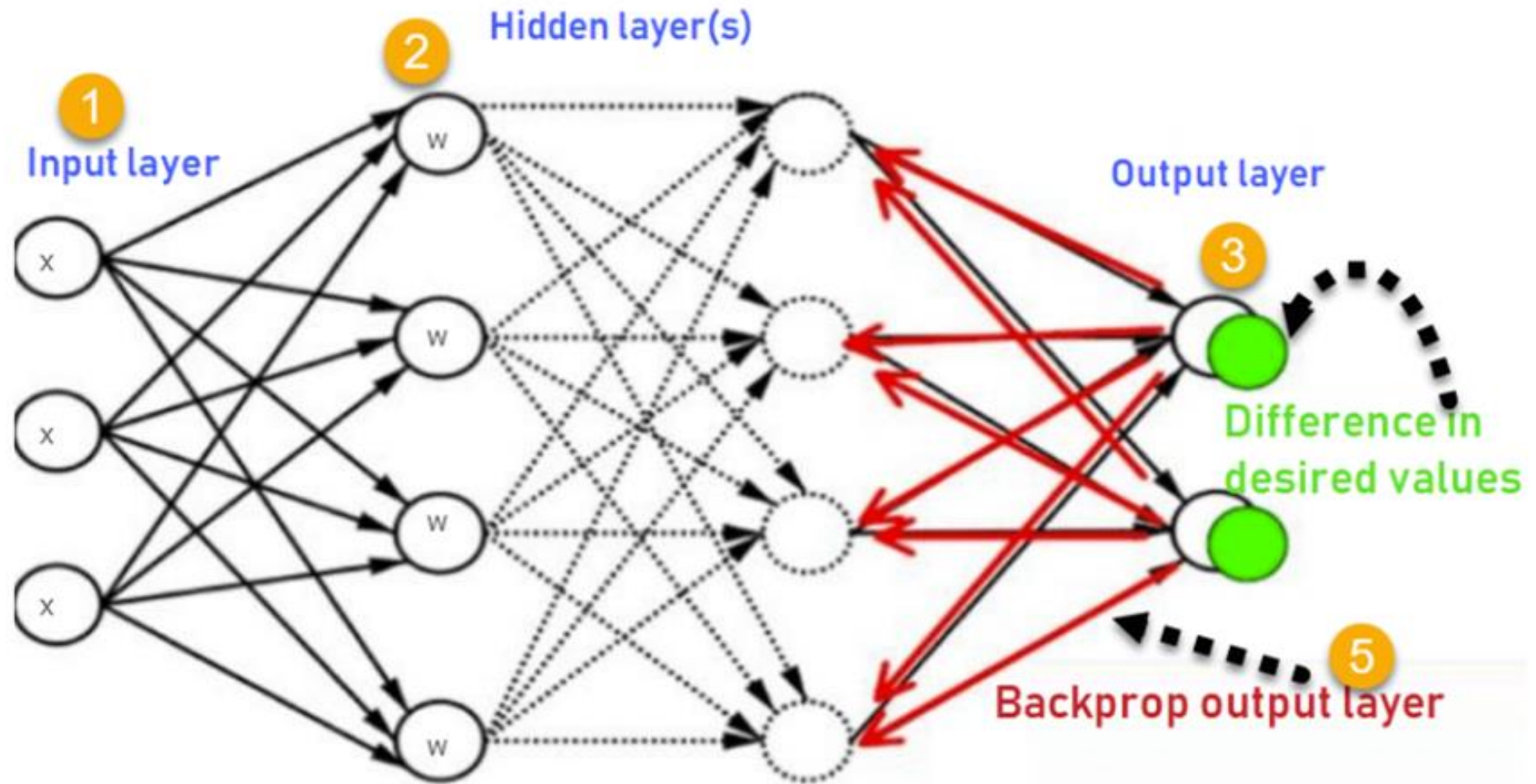
- Backpropagation is a neural network learning algorithm.
- There are many different kinds of neural networks and neural network algorithms.
- The most popular neural network algorithm is *backpropagation*, which gained *repute* in the 1980s.
- Multilayer feed-forward networks is type of neural network on which the backpropagation algorithm performs.
- Backpropagation learns for a set of weights that fits the training data so as to minimize the mean squared distance between the network's class prediction and the known target value of the tuples.

Backpropagation

- Backpropagation is the essence of neural network training.
- It is the method of fine-tuning the weights of a neural network based on the error rate obtained in the previous epoch (i.e., iteration).
- Proper tuning of the weights allows you to reduce error rates and make the model reliable by increasing its generalization.
- It is a standard method of training artificial neural networks.
- This method helps calculate the gradient of a loss function with respect to all the weights in the network

Working of Backpropagation

- Inputs X , arrive through the preconnected path
- Input is modeled using real weights W . The weights are usually randomly selected.
- Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.
- Calculate the error in the outputs
 - $\text{Error} = \text{Actual Output} - \text{Desired Output}$
- Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.
- Keep repeating the process until the desired output is achieved



Backpropagation

- Iteratively process a set of training tuples & compare the network's prediction with the actual known target value
- For each training tuple, the weights are modified to **minimize the mean squared error** between the network's prediction and the actual target value
- Modifications are made in the “**backwards**” direction: from the output layer, through each hidden layer down to the first hidden layer, hence “**backpropagation**”
- Steps
 - Initialize weights and biases in the network
 - Propagate the inputs forward (by applying activation function)
 - Backpropagate the error (by updating weights and biases)
 - Terminating condition (when error is very small, etc.)

Types of Backpropagation

- **Static back-propagation:**

- It is one kind of backpropagation network which produces a mapping of a static input for static output. It is useful to solve static classification issues like optical character recognition.

- **Recurrent Backpropagation:**

- Recurrent Back propagation in data mining is fed forward until a fixed value is achieved. After that, the error is computed and propagated backward.
- The main difference between both of these methods is: that the mapping is rapid in static back-propagation while it is nonstatic in recurrent backpropagation.

Backpropagation

- **Advantages of Backpropagation**

- Backpropagation is fast, simple and easy to program
- It has no parameters to tune apart from the numbers of input
- It is a flexible method as it does not require prior knowledge about the network
- It is a standard method that generally works well
- It does not need any special mention of the features of the function to be learned.

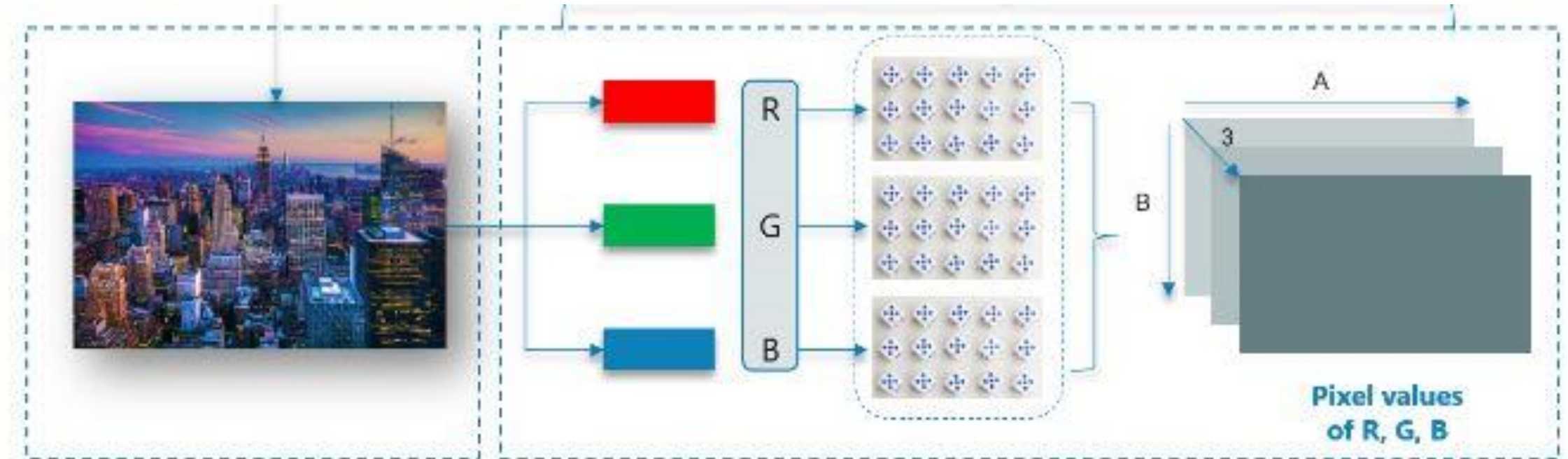
- **Disadvantages**

- The actual performance of backpropagation on a specific problem is dependent on the input data.
- Back propagation algorithm in data mining can be quite sensitive to noisy data
- You need to use the matrix-based approach for backpropagation.

Convolutional Neural Network

- CNN is essentially a classification structure for classifying images into labeled classes.
- The various layers of the CNN extract image features and finally learn to classify the images.
- Hence, the outputs of a typical CNN represent the classes or the labels of the classes
- Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers,
 - Convolutional layer
 - Pooling layer
 - Fully-connected (FC) layer

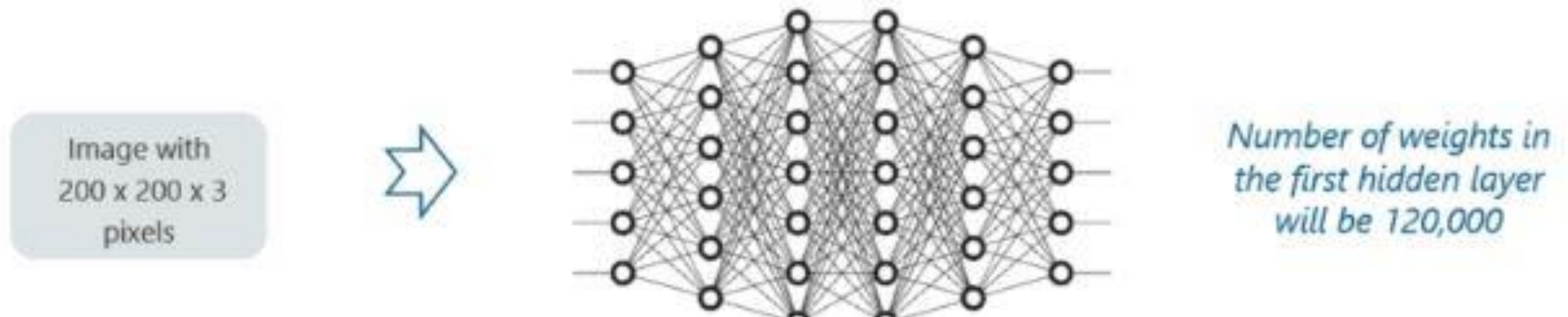
How Does A Computer Read an Image?



How Does A Computer Read an Image?

- The image is broken down into 3 color-channels which is Red, Green and Blue. Each of these color channels are mapped to the image's pixel.
- Then, the computer recognizes the value associated with each pixel and determine the size of the image.
- However, for black-white images, there is only one channel and the concept is the same.

Fully Connected Networks VS. CNN



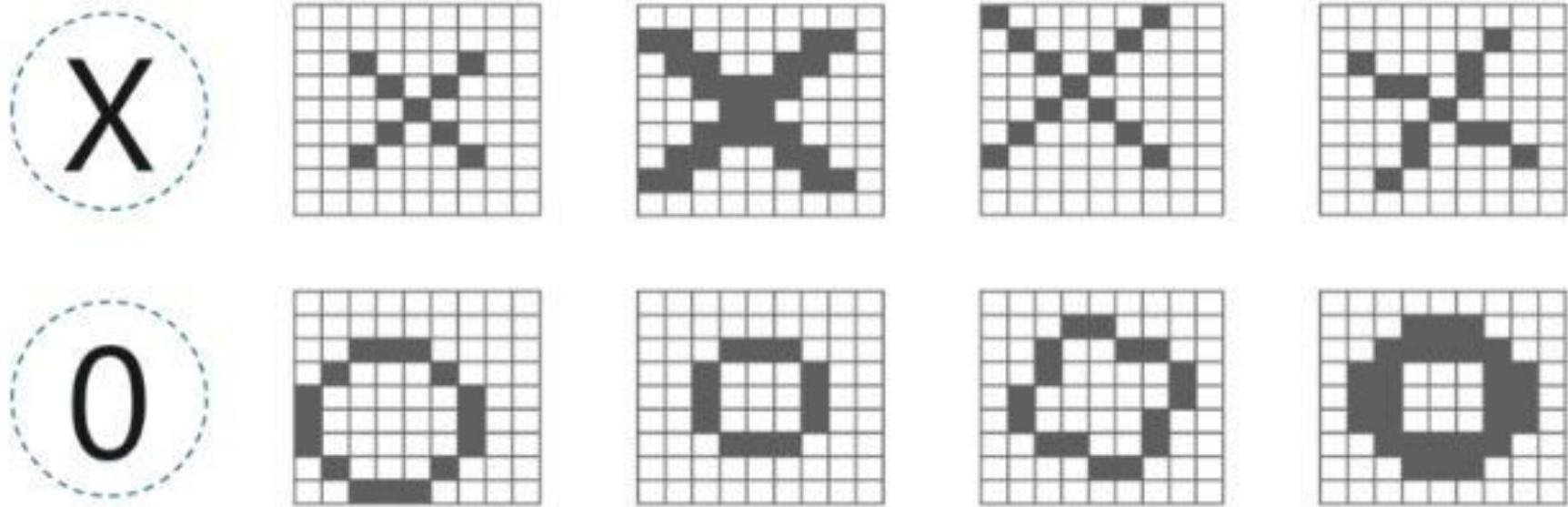
Fully Connected Networks VS. CNN

- Any generic input image will atleast have 200x200x3 pixels in size. The size of the first hidden layer becomes a whooping 120,000. If this is just the first hidden layer, imagine the number of neurons needed to process an entire complex image-set.
- This leads to over-fitting and isn't practical. Hence, we cannot make use of fully connected networks.

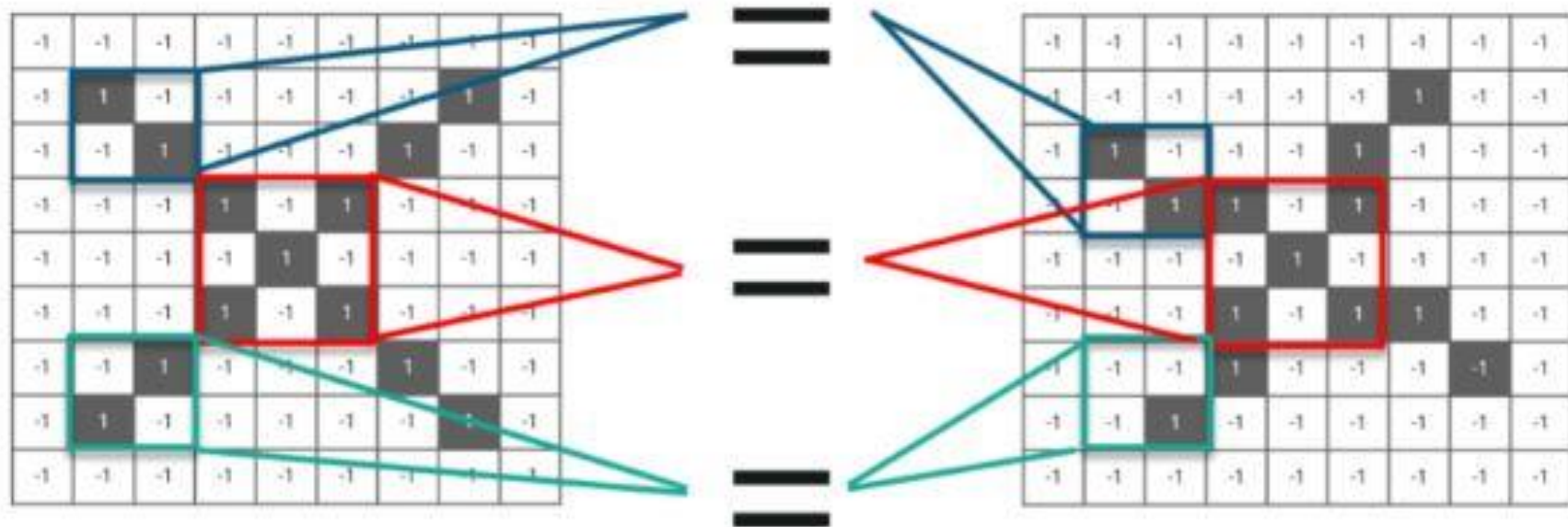
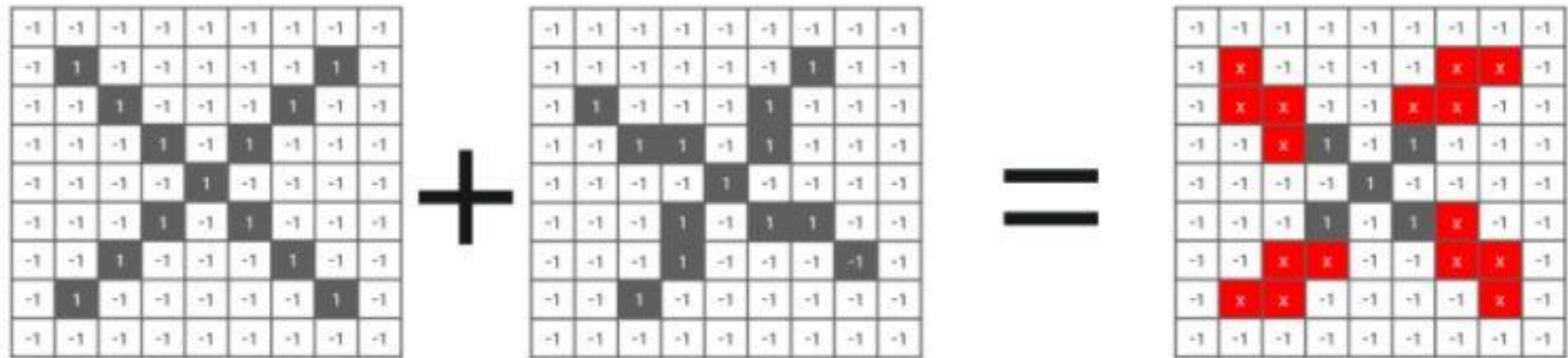
How Do Convolutional Neural Networks Work?

- There are four layered concepts we should understand in Convolutional Neural Networks:
- Convolution,
- ReLu,
- Pooling and
- Full Connectedness (Fully Connected Layer).

Example : CNN



-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



Example : CNN

- The computer understands every pixel. In this case, the white pixels are said to be -1 while the black ones are 1
- differentiate the pixels in a basic binary classification.
- search and compare the values between a normal image and another 'x' rendition, to get a lot of missing pixels.
- Take small patches of the pixels called filters and try to match them in the corresponding nearby locations to see if we get a match.
- **In this way, Convolutional Neural Network gets a lot better at seeing similarity than directly trying to match the entire image.**

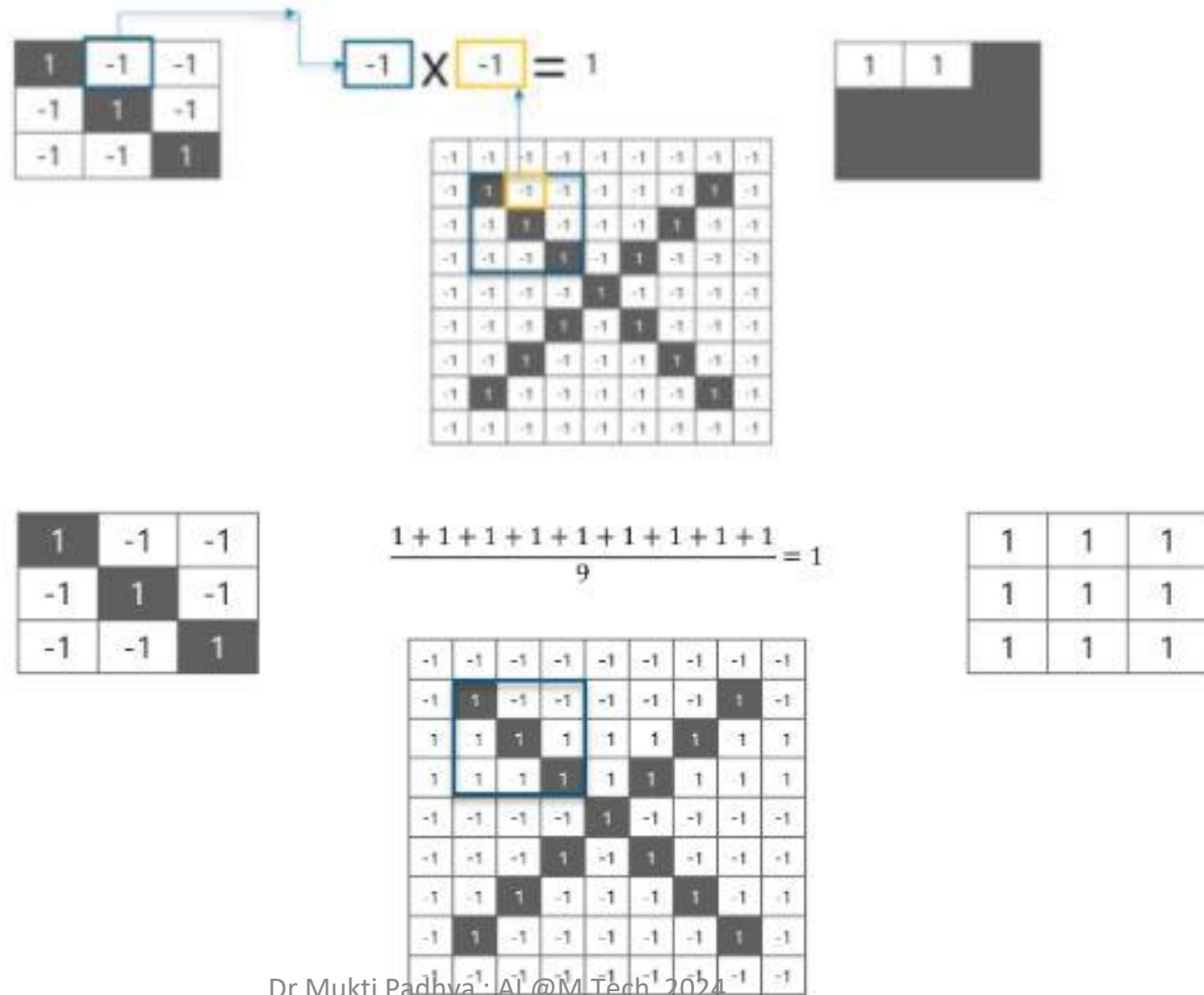
Convolutional Neural Networks

- There are four layered concepts we should understand in Convolutional Neural Networks:
- Convolution,
- ReLu,
- Pooling and
- Full Connectedness (Fully Connected Layer).

1. Convolution Of An Image

- Convolution has the nice property of being translational invariant.
- Intuitively, this means that each convolution filter represents a feature of interest (e.g pixels in letters) and the Convolutional Neural Network algorithm learns which features comprise the resulting reference (i.e. alphabet).
- We have 4 steps for convolution:
 - Line up the feature and the image
 - Multiply each image pixel by corresponding feature pixel
 - Add the values and find the sum
 - Divide the sum by the total number of pixels in the feature

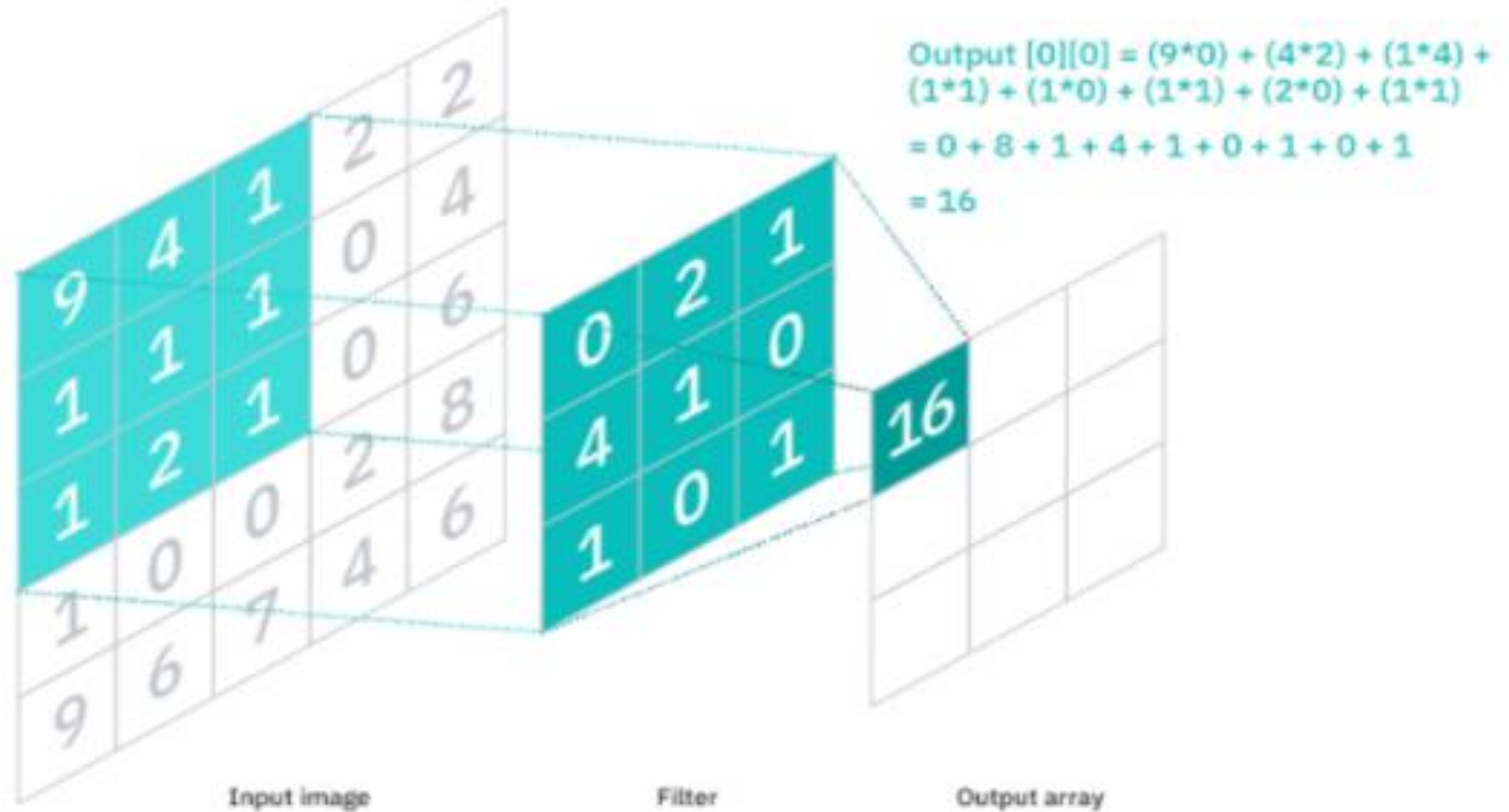
1. Convolution Of An Image



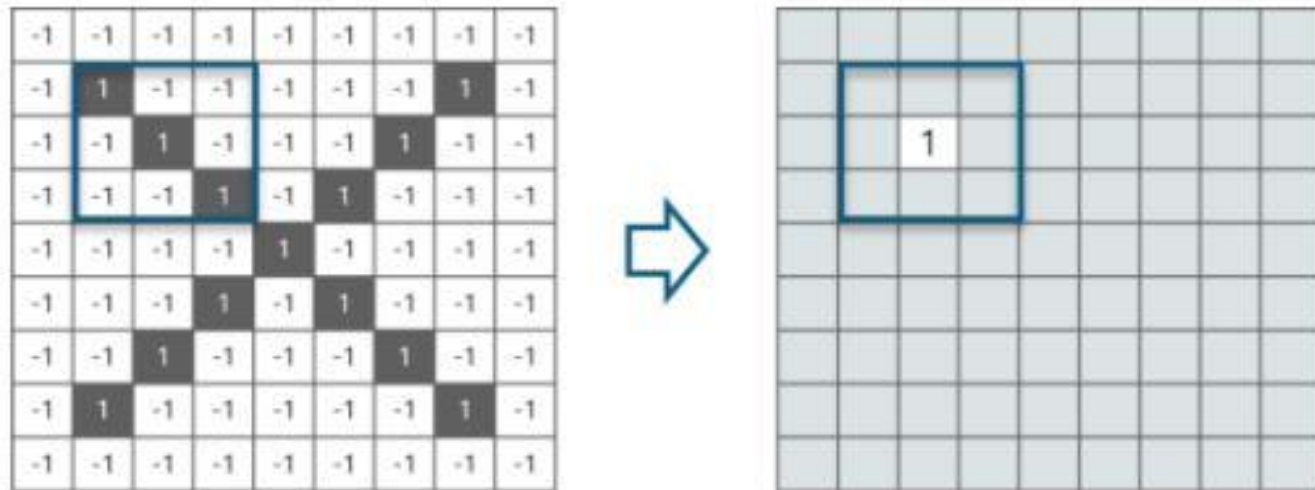
1. Convolution Of An Image

- First 2 Steps : considere a feature image and one pixel from it. We multiplied this with the existing image and the product is stored in another buffer feature
- Last 2 Steps: added the values which led to the sum. We then, divide this number by the total number of pixels in the feature image. When that is done, the final value obtained is placed at the center of the filtered image
- The output signal strength is not dependent on where the features are located, but simply whether the features are present. Hence, an alphabet could be sitting in different positions and the Convolutional Neural Network algorithm would still be able to recognize it.

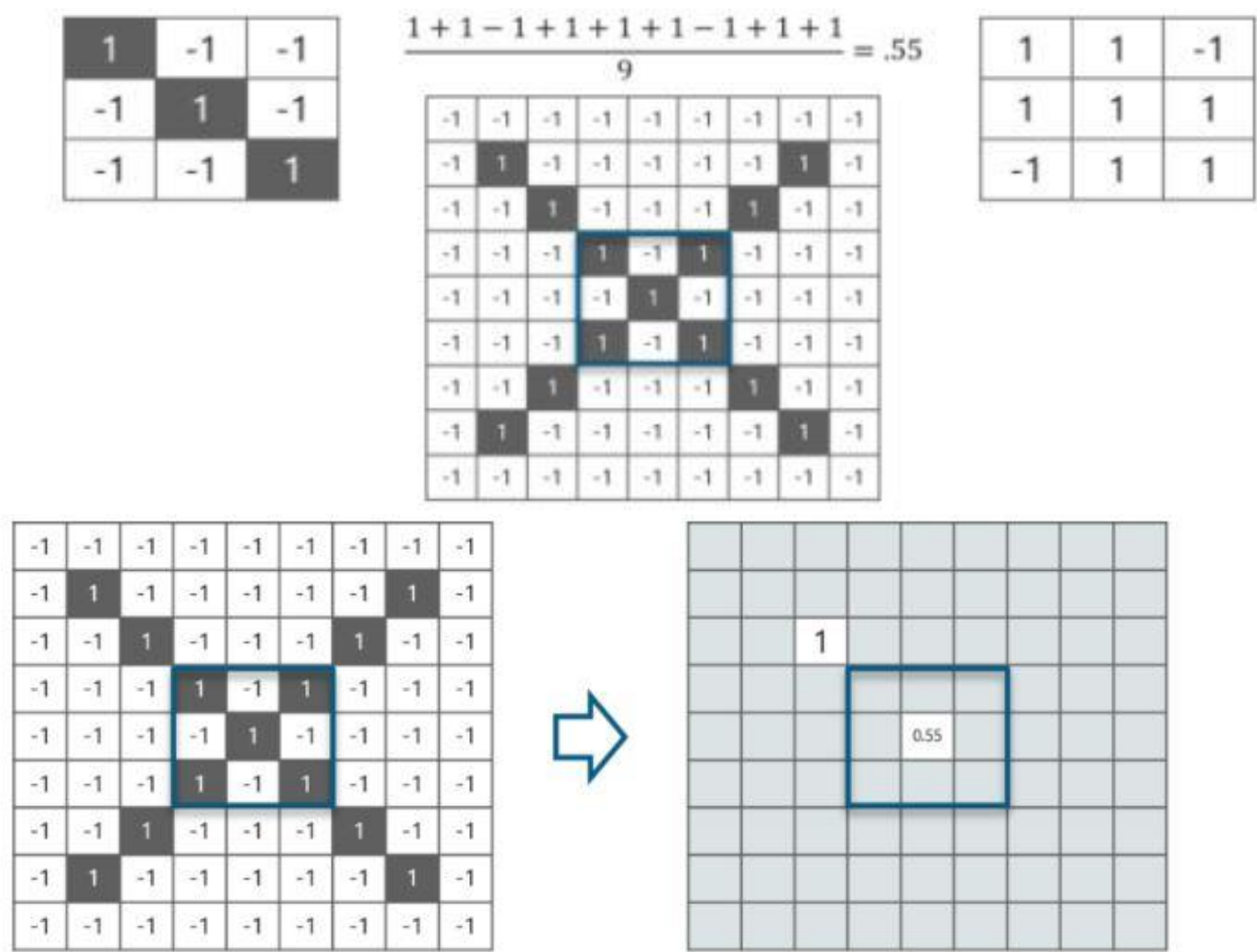
1. Convolution Of An Image



1. Convolution Of An Image



1. Convolution Of An Image



1. Convolution Of An Image

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

2. Activation Function (ReLU Layer)

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

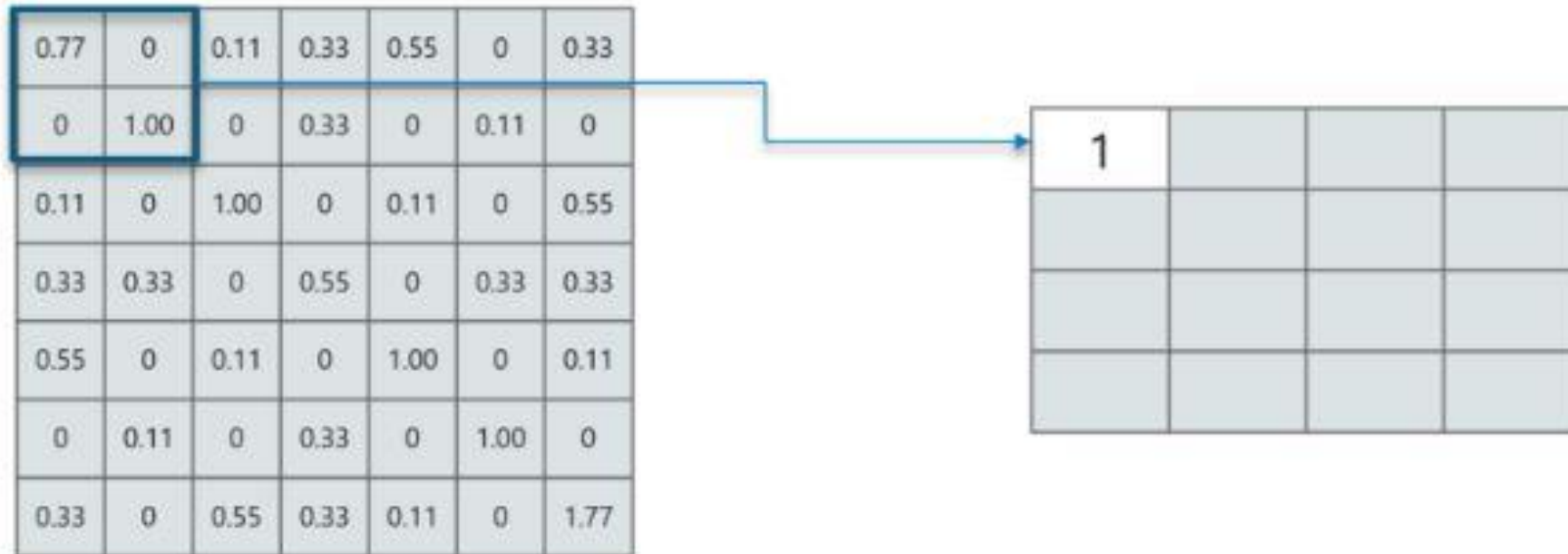


0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

2. Activation Function (ReLU Layer)

- Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable.
- The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero

3. Pooling



3. Pooling

- In this layer we shrink the image stack into a smaller size. Pooling is done after passing through the activation layer. We do this by implementing the following 4 steps:
- Pick a window size (usually 2 or 3)
- Pick a stride (usually 2)
- Walk your window across your filtered images
- From each window, take the maximum value

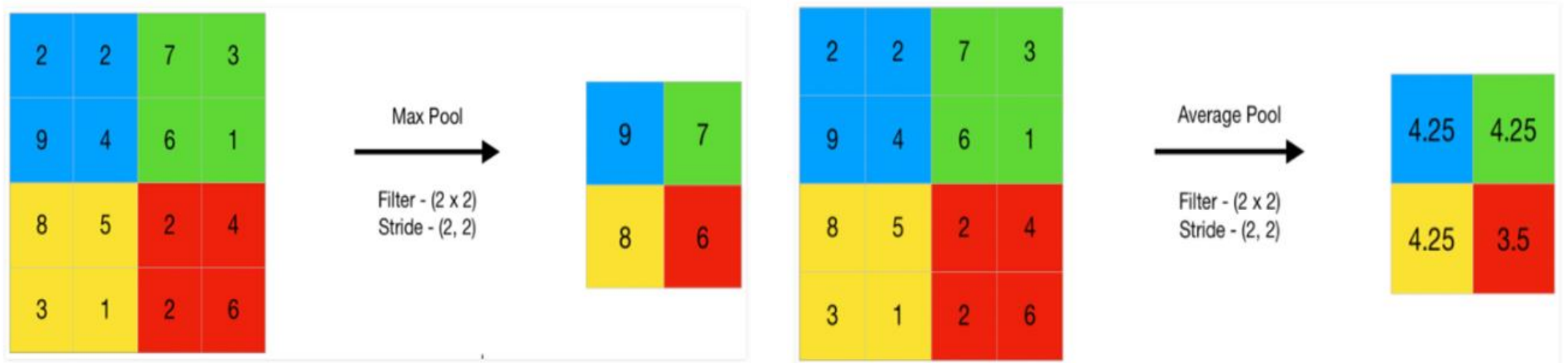
3. Pooling

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

3. Pooling

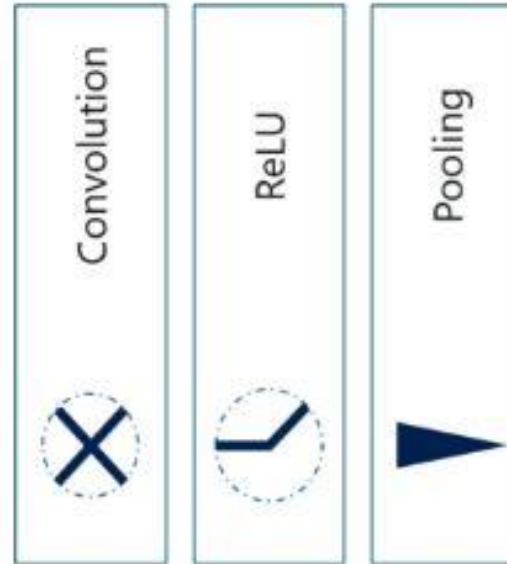


3. Pooling

- Pooling layers, also known as downsampling, conducts dimensionality reduction, reducing the number of parameters in the input.
- Similar to the convolutional layer, the pooling operation sweeps a filter across the entire input, but the difference is that this filter does not have any weights. There are two main types of pooling:
 - Max pooling: As the filter moves across the input, it selects the pixel with the maximum value to send to the output array. As an aside, this approach tends to be used more often compared to average pooling.
 - Average pooling: As the filter moves across the input, it calculates the average value within the receptive field to send to the output array.
- While a lot of information is lost in the pooling layer, it also has a number of benefits to the CNN.
- They help to reduce complexity, improve efficiency, and limit risk of overfitting.

Stacking Up The Layers

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

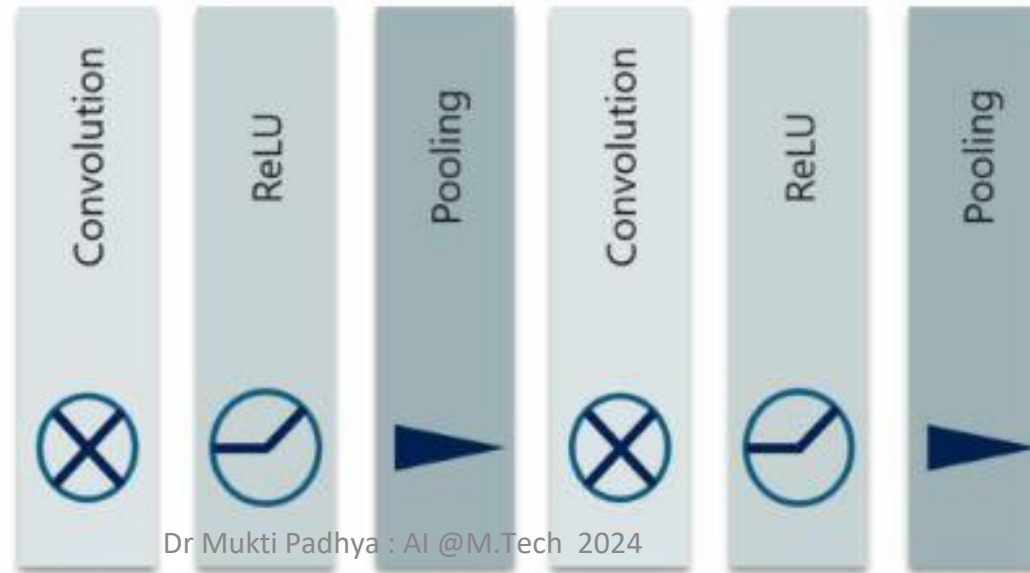


1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1



1	0.55
0.55	1.00

1	0.55
0.55	0.55

0.55	1.00
1.00	0.55

Prediction Of Image Using Convolutional Neural Networks – Fully Connected Layer

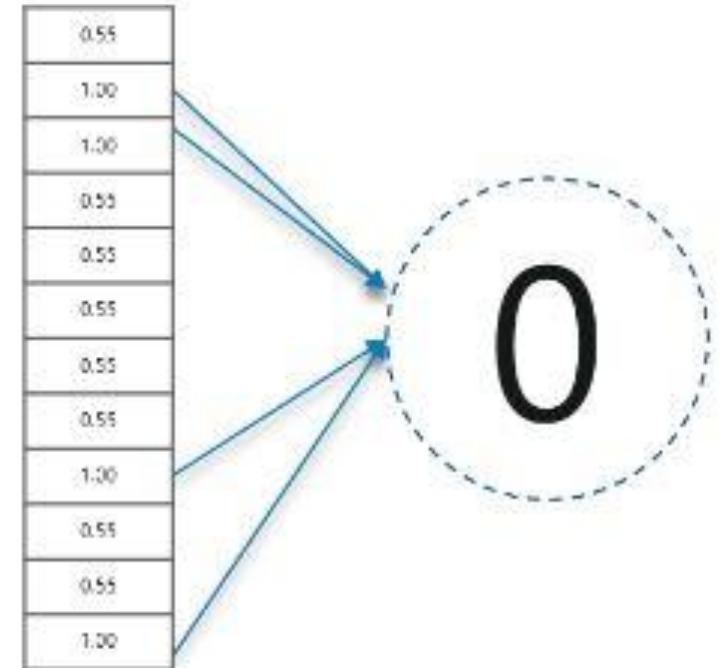
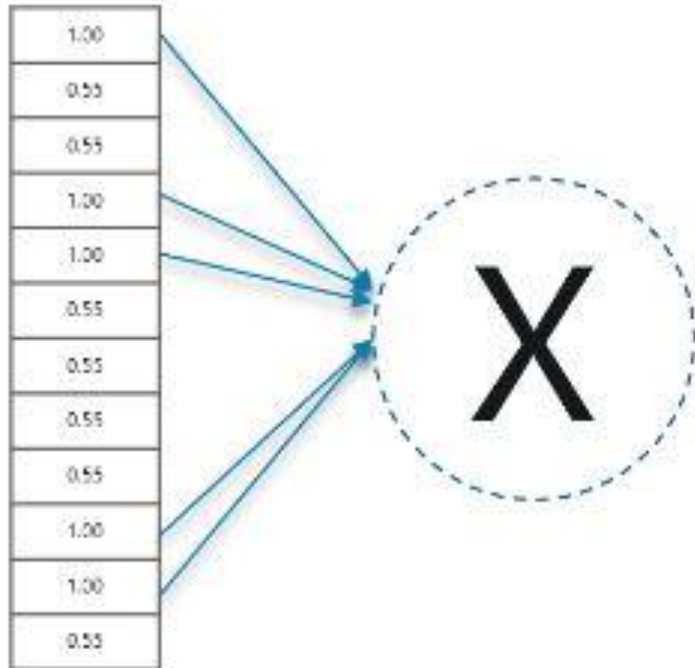
1	0.55
0.55	1.00

1	0.55
0.55	0.55

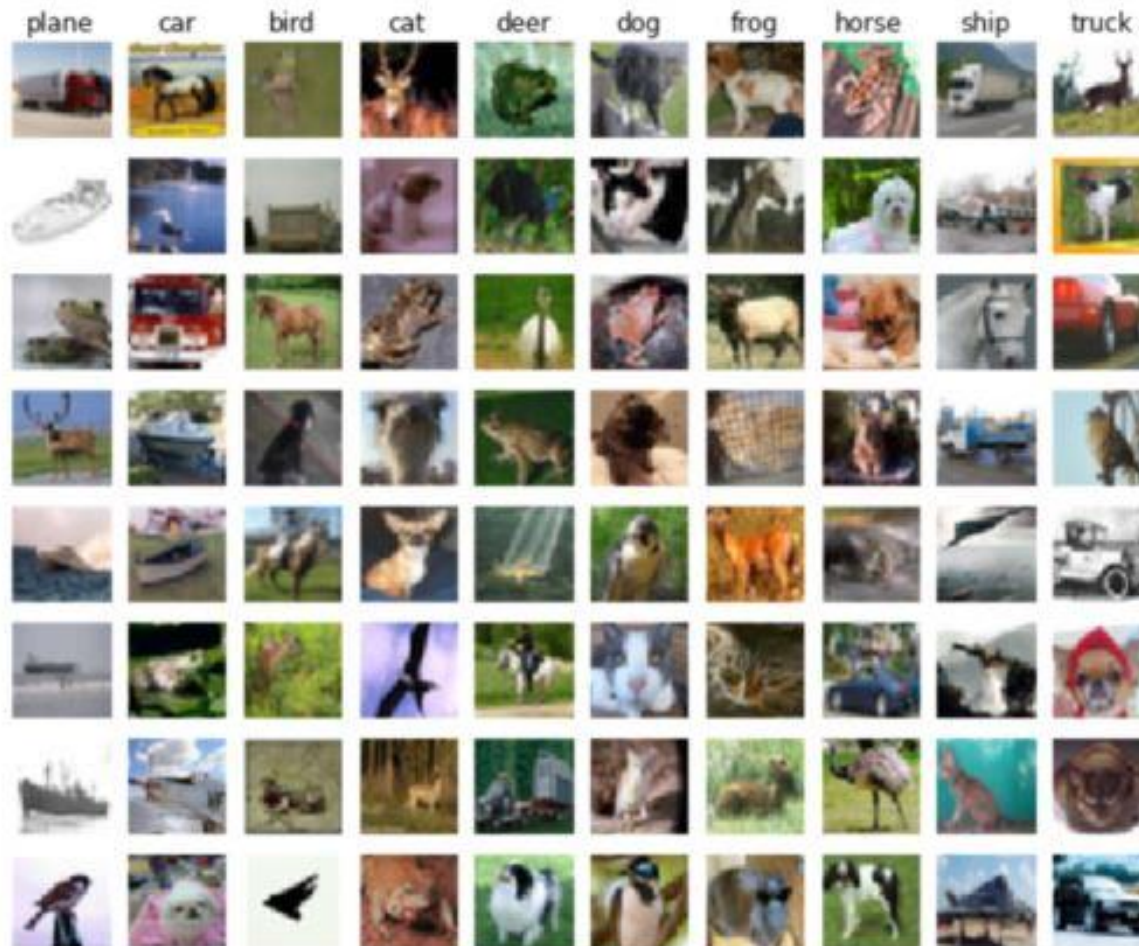
0.55	1.00
1.00	0.55

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
1.00
1.00
0.55

Prediction Of Image Using Convolutional Neural Networks – Fully Connected Layer



CNN : Image Labeling Application

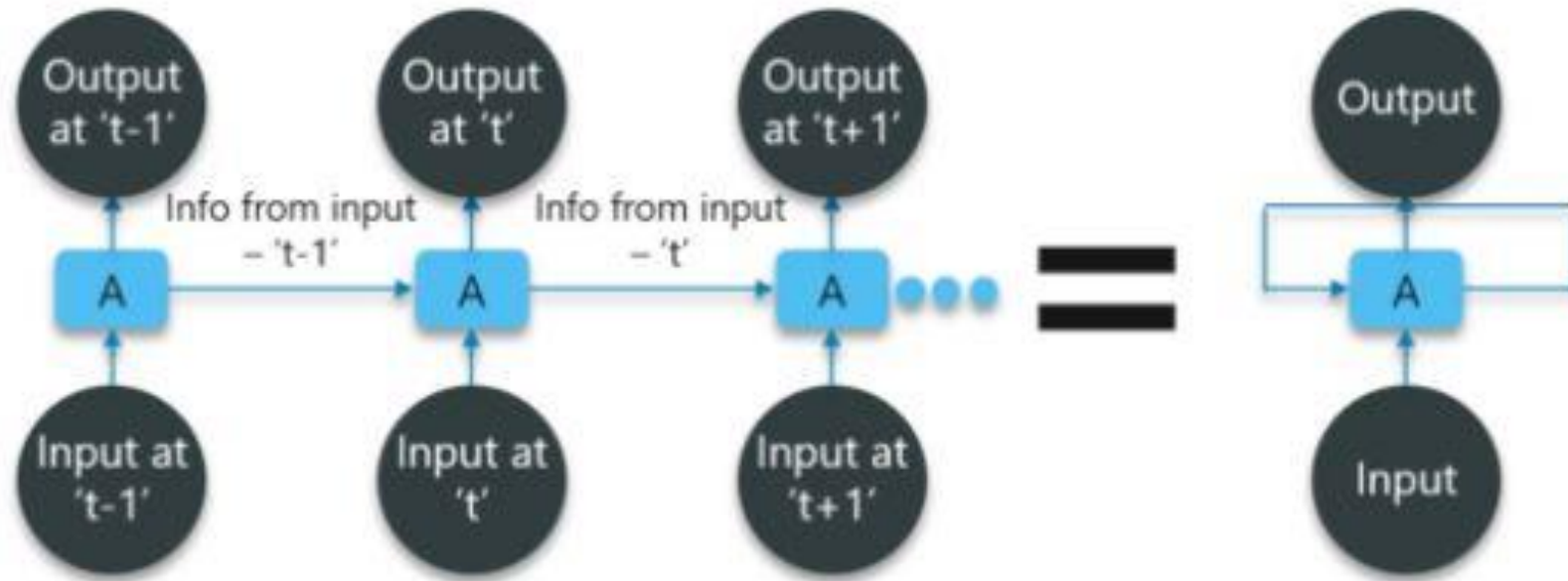


Recurrent Neural Network

- Recurrent Neural Network(RNN) are a type of Neural Network where the output from previous step are fed as input to the current step.
- In traditional neural networks, all the inputs and outputs are independent of each other, but in cases like when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.
- Thus RNN came into existence, which solved this issue with the help of a Hidden Layer.
- The main and most important feature of RNN is Hidden state, which remembers some information about a sequence.
- RNN converts the independent activations into dependent activations by providing the same weights and biases to all the layers, thus reducing the complexity of increasing parameters and memorizing each previous outputs by giving each output as input to the next hidden layer.
- Hence these three layers can be joined together such that the weights and bias of all the hidden layers is the same, into a single recurrent layer.

Recurrent Neural Network: RNN

- Recurrent Networks are a type of artificial neural network designed to recognize patterns in sequences of data, such as text, genomes, handwriting, the spoken word, numerical times series data emanating from sensors, stock markets and government agencies.



RNN

- we have certain inputs at ' $t-1$ ' which is fed into the network. These inputs will lead to corresponding outputs at time ' $t-1$ ' as well.
- At the next timestamp, information from the previous input ' $t-1$ ' is provided along with the input at ' t ' to eventually provide the output at ' t ' as well.
- This process repeats, to ensure that the latest inputs are aware and can use the information from the previous timestamp is obtained.

RNN : Scope and Limitation

Scope:

The colour of the sky is ____.



The colour of the sky is blue.

Limitation

I spent 20 long years working for the under-privileged kids in Spain. I then moved to Africa.

.....

I can speak fluent ____.

Applications of RNN

- Prediction problems
- Machine Translation
- Speech Recognition
- Language Modelling and Generating Text
- Video Tagging
- Generating Image Descriptions
- Text Summarization
- Call Center Analysis
- Face detection,
- OCR Applications as Image Recognition

Limitations of RNN

- Recurrent Neural Networks work fine when we are dealing with short-term dependencies.
- The RNN need not remember what was said before this, or what was its meaning
- RNNs fail to understand the context behind an input.
- Something that was said long before, cannot be recalled when making predictions in the present.
- The relevant information may be separated from the point where it is needed, by a huge load of irrelevant data.

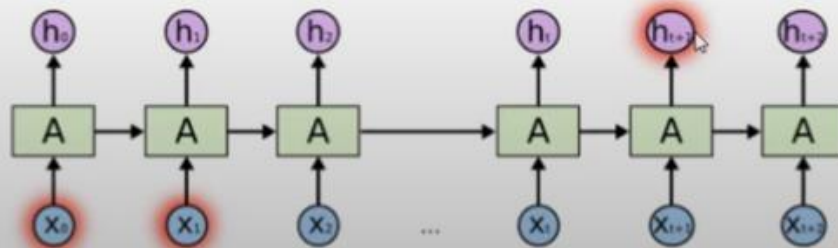
The problem with RNN

RNNs are good with short sequences such as:
The sky is _____ (Answer: blue)

But...

*This is the 10th day of wildfires in the San Francisco bay area.
There is smoke everywhere, it is snowing ash and the sky is _____.
(Correct answer: Red)*

A lot of previous information is necessary to get the answer right and RNNs seem to fail.



LSTM is a special kind of RNN

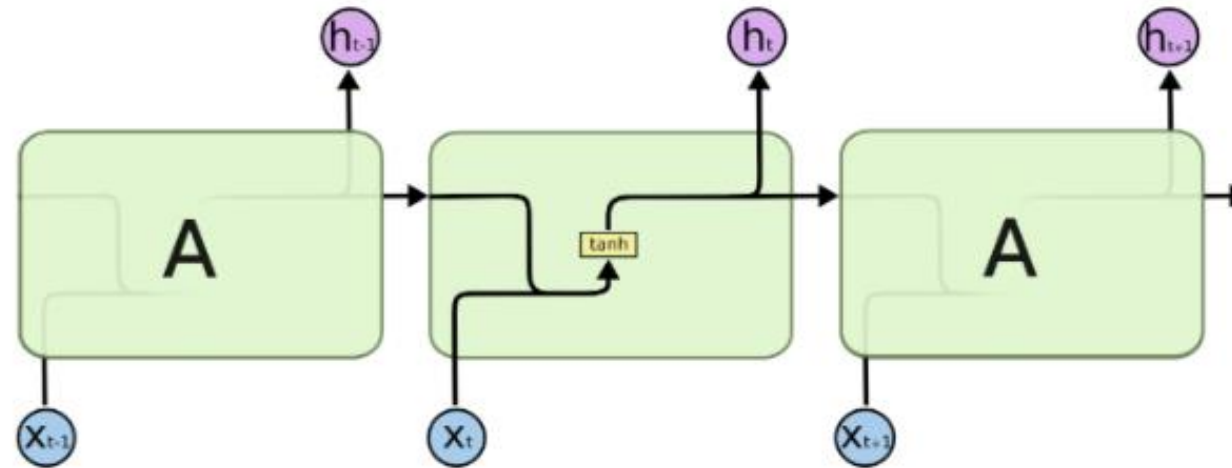
LSTM – Long Short-Term Memory

Designed to overcome limitations of RNNs such as:

- Gradient vanishing and exploding
- Complex training
- Difficulty to process very long sequences

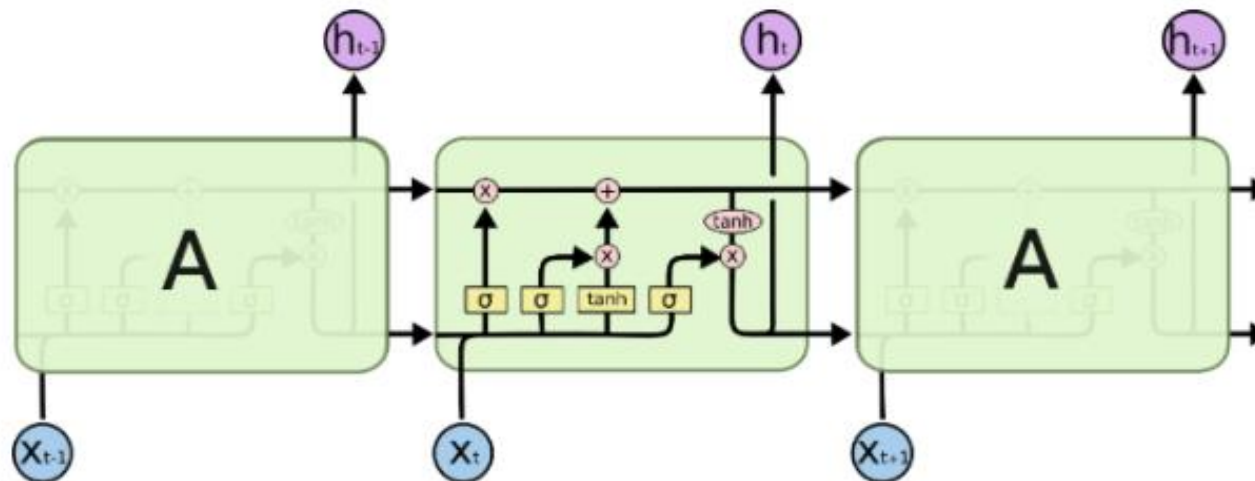
Remembering information for long periods of time is intrinsic to LSTM

RNN



The repeating module in a standard RNN contains a single layer.

LSTM



The repeating module in an LSTM contains four interacting layers.

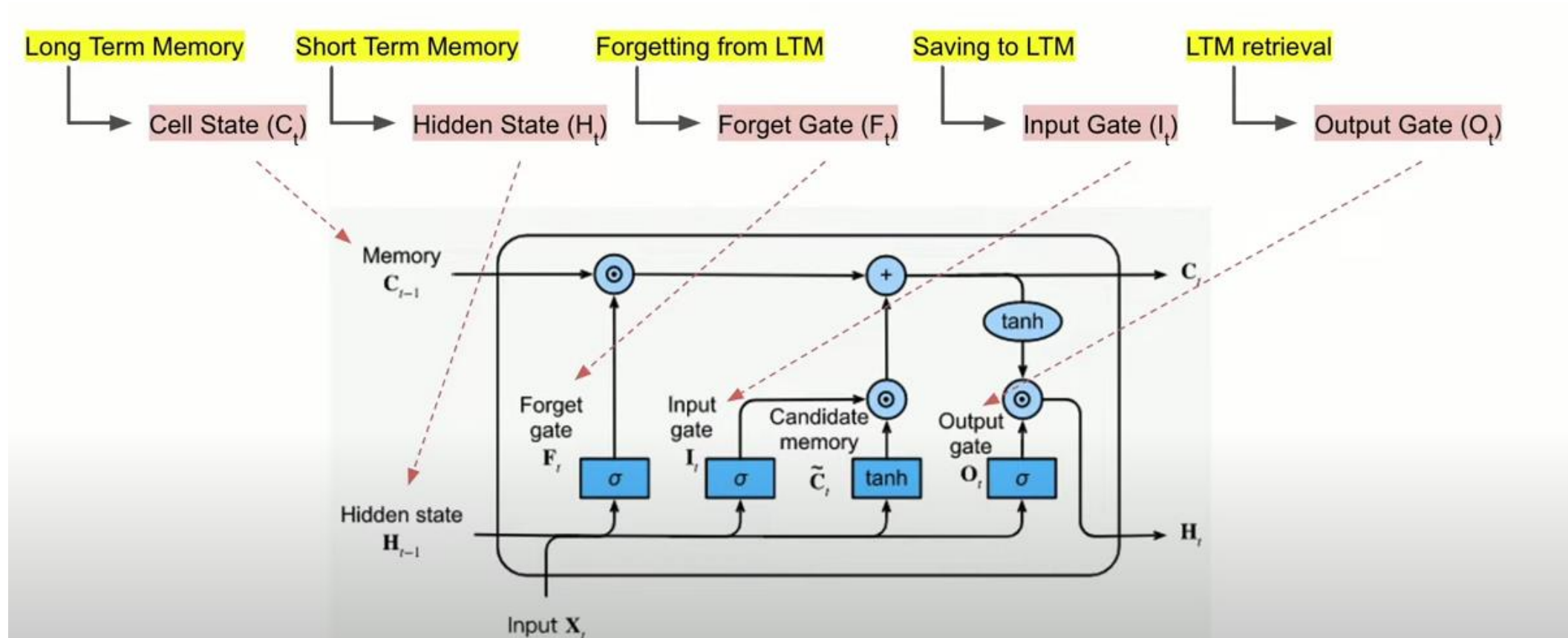
Long Short-Term Memory Network (LSTM)

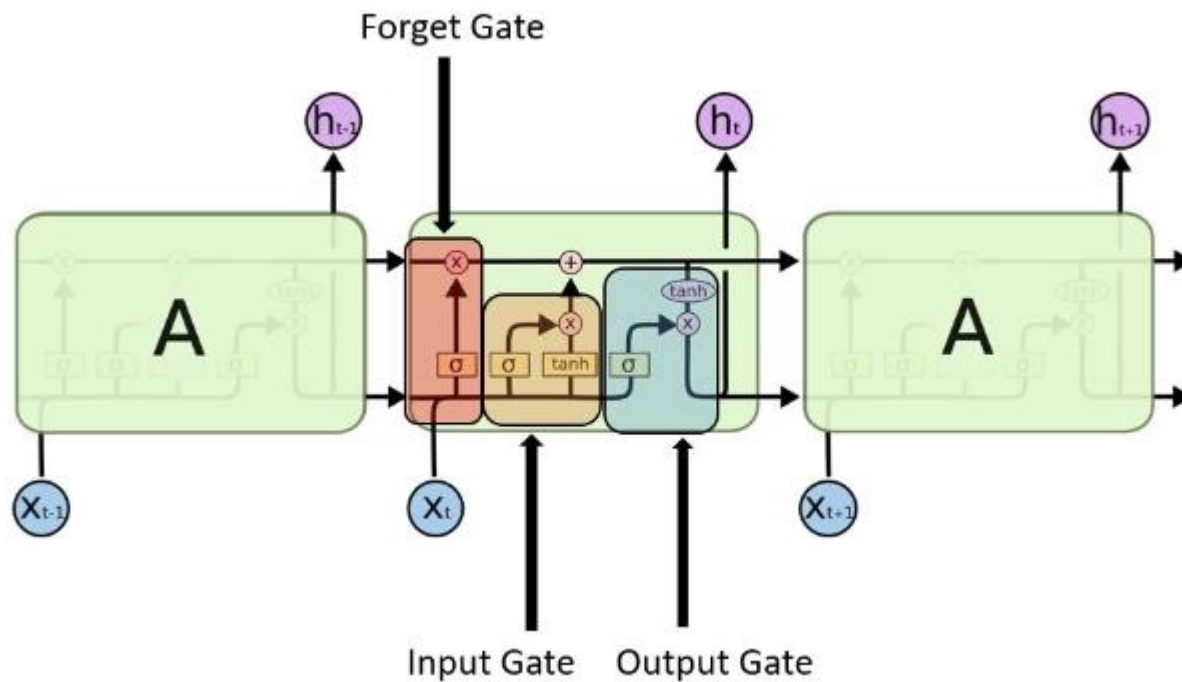
- Long Short Term Memory is a kind of recurrent neural network.
- In RNN output from the last step is fed as input in the current step.
- It tackles the problem of long-term dependencies of RNN in which the RNN cannot predict the word stored in the long-term memory but can give more accurate predictions from the recent information.
- As the gap length increases RNN does not give an efficient performance.
- LSTM can by default retain the information for a long period of time.
- It is used for processing, predicting, and classifying on the basis of time-series data.

Architecture of LSTM

- A typical LSTM network is comprised of different memory blocks called cells
- There are two states that are being transferred to the next cell; the cell state and the hidden state.
- The memory blocks are responsible for remembering things and manipulations to this memory is done through three major mechanisms, called gates
- The LSTM uses data from following:
 - The previous cell state (i.e. the information that was present in the memory after the previous time step)
 - The previous hidden state (i.e. this is the same as the output of the previous cell)
 - The input at the current time step (i.e. the new information that is being fed in at that moment)

Architecture of LSTM





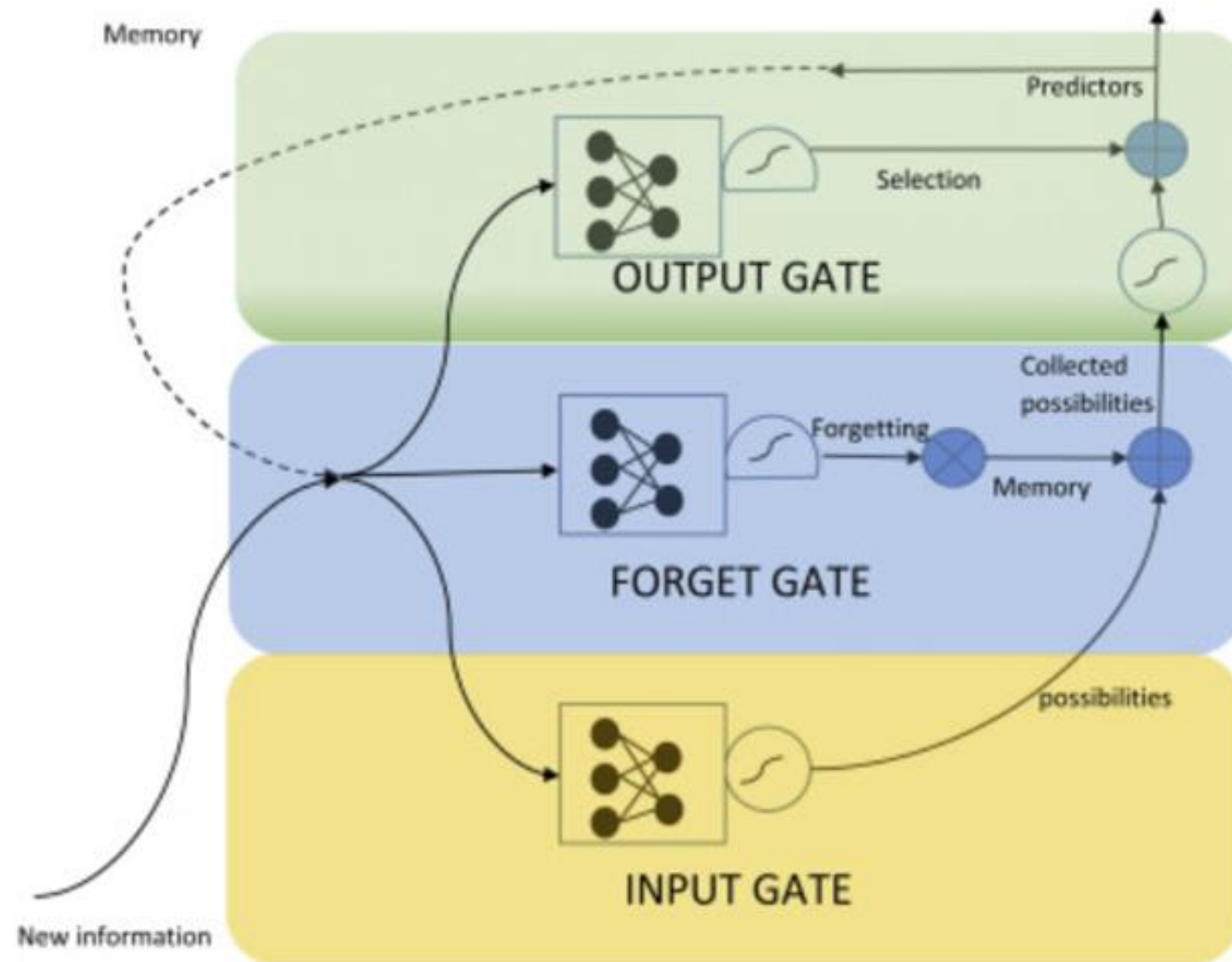
Why sigmoid?

Sigmoid can output 0 to 1, it can be used to forget or remember the information.

Why tanh?

To overcome the vanishing gradient problem. tanh's second derivative can sustain for a long range before going to zero.

States of LSTM



States and Applications of LSTM

- Forget State: LTM goes to forget gate and it forgets information that is not useful.
- Learn State: Event (current input) and STM are combined together so that necessary information that we have recently learned from STM can be applied to the current input.
- Remember State: LTM information that we haven't forget and STM and Event are combined together in Remember gate which works as updated LTM.
- Use State: This gate also uses LTM, STM, and Event to predict the output of the current event which works as an updated STM.

States of LSTM

- Forget Gate:
 - A forget gate is responsible for removing information from the cell state.
 - The information that is no longer required for the LSTM to understand things or the information that is of less importance is removed via multiplication of a filter.
 - This is required for optimizing the performance of the LSTM network.
 - The sigmoid function is responsible for deciding which values to keep and which to discard.
 - If a '0' is output for a particular value in the cell state, it means that the forget gate wants the cell state to forget that piece of information completely.
 - Similarly, a '1' means that the forget gate wants to remember that entire piece of information.
 - Example:

Bob is a nice person. Dan on the other hand is evil.

States of LSTM

- Input Gate
 - The input gate is responsible for the addition of information to the cell state.
 - This addition of information is basically three-step process
 - Regulating what values need to be added to the cell state by involving a sigmoid function. This is basically very similar to the forget gate and acts as a filter for all the information.
 - Creating a vector containing all possible values that can be added to the cell state.
 - Multiplying the value of the regulatory filter (the sigmoid gate) to the created vector (the tanh function) and then adding this useful information to the cell state via addition operation.
 - Once this three-step process is done with, we ensure that only that information is added to the cell state that is important and is not redundant.
 - Example: *Bob knows swimming. He told me over the phone that he had served the navy for 4 long years.*

States of LSTM

- Output Gate
 - Not all information that runs along the cell state, is fit for being output at a certain time.
 - This job of selecting useful information from the current cell state and showing it out as an output is done via the output gate.
 - The functioning of an output gate can again be broken down to three steps:
 - Creating a vector after applying tanh function to the cell state.
 - Making a filter using the values that it can regulate the values that need to be output from the vector created.
 - Multiplying the value of this regulatory filter to the vector created in step 1, and sending it out as a output and also to the hidden state of the next cell.
 - The filter needs to be built on the input and hidden state values and be applied on the cell state vector.
 - Example: *Bob fought single handedly with the enemy and died for his country. For his contributions brave ____.*

Applications of LSTM

- Language Modelling
- Machine Translation
- Image Captioning
- Handwriting generation
- Question Answering Chatbots